



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

Norme di progetto

Stato:	Completo
Versione:	1.3.0
Responsabile:	Riccardo Manisi
Redattori:	Ferdinando Fracasso Riccardo Manisi Gabriele Scaggiante Dennis Parolin Giovanni Visentin
Verificatori:	Dennis Parolin Marco Sanguin Giovanni Visentin Ferdinando Fracasso
Destinatari:	Gruppo BitByBit



Registro delle modifiche

Versione	Data	Autore	Descrizione	Verificatore
1.3.0	2026-04-29	Giovanni Visentin	Sistemazione convenzioni di gerarchia delle directory per i file frontend 2.2.4.1.4	Dennis Parolin
1.2.0	2026-03-31	Riccardo Manisi	Miglioramento del Processo^G di Verifica^G secondo standard ISO elencato.	Dennis Parolin
1.1.0	2026-03-31	Riccardo Manisi	Creazione sezioni per normative attività di codifica 2.2.4.1 2.2.4.2	Dennis Parolin
1.0.0	2026-03-01	Dennis Parolin	Approvazione documento.	Giovanni Visentin
0.11.5	2026-02-09	Dennis Parolin	Aggiunta una descrizione mancante.	Marco Sanguin
0.11.4	2026-02-06	Dennis Parolin	Modificata la sezione "Stesura dei requisiti" modificando la nomenclatura degli identificatori dei requisiti.	Giovanni Visentin



0.11.3	2026-02-03	Dennis Parolin	Fix riferimenti a sezioni. Aggiunti i placeholder per raggiungere sezioni esistenti ma non raggiungibili. Sistemati vari errori di battitura, errori grammaticali, refusi e errori logici. Aggiunto strumento non tracciato nella tabella degli strumenti utilizzati	Giovanni Visentin
0.11.2	2026-01-11	Dennis Parolin	Aggiunti gli strumenti di supporto delle sezioni "Processo ^G di Verifica ^G ", "Processo ^G di Validazione ^G " e "Codifica". Aggiunta la lista delle tabelle.	Riccardo Manisi
0.11.1	2026-01-10	Dennis Parolin	Sistematte le sezioni "Processo ^G di Verifica ^G " e "Processo ^G di Validazione ^G " tra i Processi di Supporto. Sistemata la sezione "Codifica" tra i Processi Primari.	Riccardo Manisi
0.11.0	2026-01-06	Dennis Parolin	Aggiunta sezioni "Processo ^G di Verifica ^G " e "Processo ^G di Validazione ^G " tra i Processi di Supporto. Aggiunta la sezione "Codifica" tra i Processi Primari.	Riccardo Manisi



0.10.1	2026-01-06	Ferdinando Fracasso	Aggiunta " Piano di qualifica^G " nella sezione Documentazione fornita	Riccardo Manisi
0.10.0	2025-12-31	Riccardo Manisi	Aggiunta sezione con metriche per QA	Ferdinando Fracasso
0.9.0	2025-12-30	Riccardo Manisi	Aggiunto Processo^G di accertamento qualità	Ferdinando Fracasso
0.8.0	2025-12-23	Riccardo Manisi	Aggiunte le sezioni di attività previste e Analisi dei requisiti^G per Processo^G di sviluppo	Ferdinando Fracasso
0.7.0	2025-12-22	Riccardo Manisi	Aggiunta attività di aggiornamento glossario	Giovanni Visentin
0.6.1	2025-12-16	Ferdinando Fracasso	Rimossa sezione ripetuta	Giovanni Visentin
0.6.0	2025-12-13	Gabriele Scaggiante	Aggiunta Processo^G di miglioramento e Processo^G di formazione	Marco Sanguin
0.5.0	2025-12-11	Ferdinando Fracasso	Aggiunta sezione " Processo^G di fornitura"	Giovanni Visentin
0.4.1	2025-12-11	Riccardo Manisi	Aggiunta prefazione con info generali del doc	Marco Sanguin
0.4.0	2025-12-10	Riccardo Manisi	Aggiunta attività di istituzione per Git e GitHub	Marco Sanguin
0.3.1	2025-12-10	Riccardo Manisi	Corretti errori grammaticali	Marco Sanguin



0.3.0	2025-12-07	Riccardo Manisi	Aggiunta attività di implementazione nel Processo^G di infrastruttura	Giovanni Visentin
0.2.0	2025-12-02	Riccardo Manisi	Aggiunta sezioni per Processo^G di configurazione e di gestione.	Marco Sanguin
0.1.1	2025-12-02	Riccardo Manisi	Sistemati errori lessicali	Marco Sanguin
0.1.0	2025-12-01	Riccardo Manisi	Creazione del documento. Aggiunto Processo^G di documentazione e bozza per le sezioni successive	Marco Sanguin



Indice

1	Introduzione	11
1.1	Scopo del documento	11
1.2	Scopo del prodotto	11
1.3	Glossario	11
1.4	Riferimenti	12
1.4.1	Riferimenti normativi	12
1.4.2	Riferimenti informativi	12
2	Processi primari	13
2.1	Fornitura	13
2.1.1	Strumenti a supporto	13
2.1.2	Attività previste	14
2.1.3	Contatti con l'azienda Proponente	14
2.1.4	Documentazione fornita	15
2.2	Processo di sviluppo	16
2.2.1	Strumenti a supporto	16
2.2.2	Attività previste	17
2.2.3	Analisi Dei Requisiti	18
2.2.3.1	Stesura dei casi d'uso	18
2.2.3.2	Stesura dei requisiti	19
2.2.4	Codifica	20
2.2.4.1	Stile di Codifica: Dart	20
2.2.4.1.1	Convenzioni di nomenclatura Dart	20
2.2.4.1.2	Convenzioni nomenclatura delle classi Dart	21
2.2.4.1.3	Formattazione automatica del codice Dart	21
2.2.4.1.4	Convenzioni sulla struttura gerarchica delle directory	21
2.2.4.2	Stile di Codifica: Python	22
2.2.4.2.1	Convenzioni di nomenclatura Python	22
2.2.4.2.2	Convenzioni sui suffissi delle classi Python	23
2.2.4.2.3	Formattazione automatica del codice Python	23
2.2.4.3	Vincoli implementativi	23
3	Processi di supporto	24
3.1	Documentazione	24
3.1.1	Obiettivi	24
3.1.2	Strumenti a supporto	25
3.1.3	Tipologie di documenti	25



3.1.4	Implementazione di Processo	26
3.1.4.1	Creazione delle Issue GitHub	26
3.1.4.2	Assegnazione della Issue	26
3.1.4.3	Impostazione del documento	26
3.1.4.4	Date	29
3.1.4.5	Modifiche a documenti	30
3.1.4.6	Aggiornamento del glossario	31
3.1.4.7	Conclusione modifiche	31
3.1.4.8	Verifica	31
3.1.4.9	Approvazione e Pull Request	31
3.1.4.10	Pubblicazione	31
3.2	Processo di gestione della configurazione	31
3.2.1	Strumenti a supporto	32
3.2.2	Attività previste	32
3.2.3	Identificazione della configurazione	32
3.2.4	Controllo della configurazione	33
3.2.4.1	Gestione Issue e pianificazione del lavoro	33
3.2.4.2	Creazione della Pull Request	34
3.2.4.3	Verifica e revisione della Pull Request	34
3.2.4.4	Integrazione nella Baseline	34
3.2.5	Registro dello stato della configurazione	34
3.2.6	Valutazione della configurazione	35
3.3	Processo di accertamento della qualità	35
3.3.1	Implementazione di Processo	35
3.3.2	Accertamento della qualità di prodotto	36
3.3.3	Accertamento della qualità di processo	36
3.4	Processo di Verifica	36
3.4.1	Strumenti a supporto	36
3.4.2	Attività previste	37
3.4.3	Implementazione di processo	37
3.4.4	Verifica documentale	37
3.4.5	Verifica Software	38
3.4.5.1	Analisi Statica	38
3.4.5.2	Analisi Dinamica (Testing)	38
3.4.6	Classificazione dei Test	39
3.5	Processo di Validazione	39
3.5.1	Obiettivi del Processo	40
3.5.2	Tracciamento come strumento di Validazione	40
3.5.3	Esecuzione dei Test Di Accettazione	40
3.5.4	Strumenti a supporto della Validazione	40



3.5.4.1	Strumenti per il tracciamento	40
3.5.4.2	Strumenti per l'esecuzione e il reporting	41
4	Processi organizzativi del Ciclo Di Vita	41
4.1	Gestione dei processi	41
4.1.1	Strumenti a supporto	42
4.1.2	Implementazione di Processo	42
4.1.3	Ruoli	43
4.1.4	Struttura dei team GitHub	44
4.1.5	Coordinamento	44
4.1.5.1	Riunioni	44
4.2	Infrastruttura	45
4.2.1	Attività previste	45
4.2.2	Implementazione	45
4.2.3	Istituzione	47
4.2.3.1	Discord	47
4.2.3.2	Git	47
4.2.3.3	GitHub	48
4.3	Processo di miglioramento	48
4.3.1	Scopo	49
4.3.2	Descrizione	49
4.3.3	Implementazione	49
4.4	Processo di formazione	49
4.4.1	Scopo	49
4.4.2	Descrizione	50
4.4.3	Aspettative formative	50
4.4.4	Piano di formazione	50
5	Metriche	51
5.1	Nomenclatura	51
5.2	Metriche di Prodotto	51
5.2.1	Metriche statiche	52
5.2.1.1	Requisiti obbligatori soddisfatti	52
5.2.1.2	Requisiti desiderabili soddisfatti	52
5.2.1.3	Requisiti opzionali soddisfatti	52
5.2.1.4	Reliability Rating (RR)	52
5.2.1.5	Maintainability Rating	53
5.2.1.6	Cyclomatic Complexity (CYC)	53
5.2.1.7	Cognitive Complexity (COC)	53
5.2.2	Metriche dinamiche	54
5.2.2.1	Code Coverage (CC)	54

5.2.2.2	Unit Test Success Density (UTSD)	54
5.3	Metriche di Processo	54
5.3.1	Fornitura	54
5.3.1.1	Budget at Completion (BAC)	54
5.3.1.2	Planned Value (PV)	55
5.3.1.3	Actual Cost (AC)	55
5.3.1.4	Earned Value (EV)	55
5.3.1.5	Schedule Performance Index (SPI)	55
5.3.1.6	Cost Performance Index (CPI)	55
5.3.1.7	Estimate at Completion (EAC)	56
5.3.1.8	Estimate to Complete (ETC)	56
5.3.2	Documentazione	56
5.3.2.1	Indice di Gulpease	56
5.3.3	Gestione dei processi	56
5.3.3.1	Sprint Commitment Reliability (SCR)	56

Elenco delle tabelle

2	Esempio di tabella dello storico di un documento	27
3	Descrizione dei ruoli del progetto e delle loro responsabilità	43
4	Strumenti utilizzati per l'infrastruttura	47



1. Introduzione

1.1. Scopo del documento

Il presente documento è volto a definire i processi del **Way of working^G** per il gruppo BitByBit che si impegna a mantenerlo per tutta la durata del progetto. La sua redazione si ispira, a scopo puramente informativo, allo standard ISO/IEC 12207:1995, dal quale vengono riprese alcune classificazioni dei **Processo^G**, senza assumerlo come riferimento normativo. In particolare, lo standard individua tre tipologie di **Processo^G**:

1. **processi primari:** atti alla produzione di un prodotto software;
2. **processi di supporto:** supportano altri processi come parte integrante e contribuiscono al successo e alla qualità del prodotto software;
3. **processi organizzativi:** definiscono i metodi organizzativi del gruppo, per stabilire e implementare la struttura costituita dai processi di **Ciclo di vita^G**.

Ogni membro del gruppo si impegna a visionare costantemente il presente documento e a rispettare rigorosamente i processi che vengono esposti di seguito, per ottenere un prodotto che sia professionale, coerente e uniforme.

1.2. Scopo del prodotto

L'azienda **Miriade S.r.l.** ha proposto lo sviluppo di un'applicazione mobile, denominata "L'app che Protegge e Trasforma" finalizzata alla prevenzione e al supporto delle vittime di violenze di genere.

Il prodotto punta a stabilirsi come uno strumento intelligente e sicuro che aiuta l'utilizzatore a riconoscere segnali di pericolo e a fornire risorse per tutelarsi da atti, che siano fisici o psicologici, che potrebbero nuocere alla sua salute. L'applicazione implementa metodologie volte alla tutela delle persone che sono già vittima di violenze, ma anche per la valutazione e l'individuazione di situazioni che potrebbero sfociare in atti di violenza.

1.3. Glossario

I documenti prodotti nei processi di **Ciclo di vita^G** sono corredati da un glossario che costituisce un riferimento condiviso. Il suo scopo è ridurre le ambiguità lessicali che possono emergere a causa dei diversi livelli di competenze tecniche tra i destinatari dei documenti. Poiché le Norme di Progetto, l'**Analisi dei requisiti^G**, il **Piano di progetto^G** e il **Piano di qualifica^G** sono rivolti a figure eterogenee, tra cui acquirenti, fornitori, sviluppatori, gestori e utenti del prodotto software, è necessario che ogni termine potenzialmente ambiguo, tecnico o soggetto a interpretazioni multiple sia riportato in modo chiaro e univoco.



Per favorire la reperibilità dei termini e rendere immediata la loro identificazione all'interno dei documenti, ogni voce del glossario è richiamata nel testo mediante la seguente convenzione stabilita dal gruppo BitByBit:

termine_nel_glossario^G

Il glossario è considerato parte integrante del **Processo^G** di documentazione: viene aggiornato ogni volta che emergono nuovi termini rilevanti, che vengono introdotti nuovi concetti tecnici o che si rende necessario chiarire ambiguità riscontrate durante la redazione o la **Verifica^G** dei documenti.

1.4. Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato^G d'appalto C4: *L'app che Protegge e Trasforma***
Parti consultate: documento completo
Versione: 1.0
Ultimo accesso: 2026-01-12
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C4.pdf>
- **ISO/IEC 12207:1997 - Software Life Cycle Processes**
Parti consultate: documento completo
Versione: edizione 1997
Ultimo accesso: 2025-12-30
https://www.math.unipd.it/~tullio/IS-1/2009/Approfondimenti/ISO_12207-1995.pdf

1.4.2 Riferimenti informativi

- **Ian Sommerville, *Software Engineering***
Parti consultate: Capitoli 2, 24
Versione: *X Edizione*
Ultimo accesso: 2026-01-12
- **Effective Dart**
Parti consultate: pagina web intera
Versione: *Page last updated on 2026-02-05*
Ultimo accesso: 2026-03-31
<https://dart.dev/effective-dart/style>
- **Architecture Flutter**
Parti consultate: architecture concepts, architecture case study
Versione: *Page last updated on 2025-09-05*



Ultimo accesso: 2026-03-31

<https://docs.flutter.dev/app-architecture>

- **Glossario**

Parti consultate: documento completo

Versione: v.1.0.0

Ultimo accesso: 2026-03-30

https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/Glossario.pdf

2. Processi primari

La presente sezione espone i seguenti processi primari:

- (a) **Fornitura;**
- (b) **Sviluppo;**

2.1. Fornitura

Il **Processo^G** primario di fornitura definisce le attività che devono essere intraprese dal fornitore. Il **Processo^G** consta dell'analisi e pianificazione delle risorse necessarie per garantire l'efficienza e la conformità del progetto ai requisiti discussi con la **Proponente^G**.

2.1.1 Strumenti a supporto

Per svolgere le attività previste dal **Processo^G** di fornitura il gruppo utilizza gli strumenti elencati di seguito.

- **Gmail:** comunicazione asincrona con la **Proponente^G** e i professori.
- **Google Chat:** comunicazione asincrona con la **Proponente^G**.
- **Discord:** pianificazione e svolgimento delle riunioni interne.
- **Google Docs:** redazione di documenti preliminari da utilizzare come base informativa per le riunioni interne.
- **GitHub Pages:** servizio di hosting statico integrato in GitHub, utilizzato per la pubblicazione automatica della documentazione prodotta dal gruppo



2.1.2 Attività previste

Le attività identificate dal gruppo per il **Processo^G** di Fornitura sono:

- **Inizializzazione:** Il fornitore effettua una **Analisi dei requisiti^G** nelle richieste del **Proponente^G**, tenendo conto di eventuali vincoli e valutando la propria capacità di realizzare la proposta, valutando possibili requisiti da ritrattare con il **Proponente^G**.
- **Pianificazione della risposta:** Il fornitore definisce una proposta in risposta alla richiesta del **Proponente^G**, tenendo conto dei risultati della fase precedente.
- **Contrattazione:** Il fornitore presenta la propria risposta, definita nell'attività precedente, al **Proponente^G** al fine di stipulare un accordo per la fornitura del progetto finale.
- **Pianificazione:** Il fornitore stabilisce un **Framework^G** per la gestione del progetto e per garantire la qualità del prodotto finale, e, nel caso non fosse definito negli accordi con il **Proponente^G**, il fornitore sceglie anche un modello di **Ciclo di vita^G** del software appropriato per il progetto. Il fornitore inoltre si occupa anche di individuare le risorse e tecnologie necessarie per il progetto, considerandone anche i relativi rischi.
- **Esecuzione e controllo:** Il fornitore esegue e implementa il piano definito nella fase precedente, monitorandone progresso e qualità del prodotto durante il **Ciclo di vita^G** precedentemente definito, interfacciandosi con il **Proponente^G** quando necessario.
- **Revisione e Validazione^G:** Il fornitore coordina attività di revisione con il **Proponente^G**, anche durante lo sviluppo del prodotto poiché necessario per avere feedback per piani futuri, ed effettua le necessarie attività di **Verifica^G** e **Validazione^G** per garantire la qualità del prodotto.
- **Consegna e completamento:** Il fornitore consegna il prodotto finale al **Proponente^G**, fornendo inoltre il necessario supporto.

2.1.3 Contatti con l'azienda Proponente

La comunicazione avverrà principalmente tramite uno spazio su Google Chat messo a disposizione dal **Proponente^G Miriade S.r.l.**, al fine di permettere una comunicazione più rapida in caso di eventuali dubbi o domande. Inoltre il **Proponente^G** si rende disponibile a incontri da remoto, tramite Google Meet, o in presenza, quando è opportuna la **Verifica^G** dello stato di avanzamento del progetto.



2.1.4 Documentazione fornita

Questa sezione elenca e descrive brevemente tutta la documentazione che verrà prodotta dal gruppo BitByBit che verrà consegnata al **Proponente^G** Miriade S.r.l. e ai committenti Prof. Tullio Vardanega e Prof. Riccardo Cardin.

- **Lettera di presentazione**

- Documento in cui il gruppo **BitByBit** formalizza la propria candidatura al **Capitolato^G** dell'azienda **Miriade S.r.l.**

Il documento contiene una versione riassunta delle ragioni per la scelta del **Capitolato^G**, un resoconto generale degli incontri preliminari effettuati con delle aziende proponenti e informazioni riguardanti il budget e la data di consegna prevista del progetto.

- **Valutazione dei capitolati**

- Documento in cui viene fatta una analisi dettagliata di ciascun **Capitolato^G** reso disponibile dai proponenti, valutandone punti a favore e punti critici, con maggiore attenzione posta sul **Capitolato^G** scelto dal gruppo.

- **Valutazione di costi e rischi**

- Documento in cui il gruppo **BitByBit** ha svolto una analisi preliminare dei rischi durante lo svolgimento del progetto e descritto dei possibili metodi di mitigazione di tali rischi.

Il documento inoltre contiene una analisi dei ruoli richiesti dal progetto e delle ore assegnate per ciascun ruolo ai membri del gruppo, il calcolo del preventivo costi per lo svolgimento del progetto, e la data di consegna prevista per il prodotto finale.

- **Analisi dei requisiti^G**

- Documento che ha l'obiettivo di analizzare in modo esaustivo i requisiti del prodotto del progetto, mediante l'identificazione dei casi d'uso, classificandoli in base alla loro importanza come obbligatori, desiderabili o opzionali.

Questo in modo da avere un punto di riferimento primario per le attività di progettazione, sviluppo, **Verifica^G** e **Validazione^G**, e garantire la conformità del prodotto finale alle richieste del **Proponente^G**.

- **Norme di progetto**

- Il presente documento, avente l'obiettivo di definire i processi del **Way of working^G** e le norme che il gruppo **BitByBit** si impegna a seguire per l'intera durata del progetto.



- **Piano di progetto^G**

- Documento in cui viene definita la pianificazione strategica e operativa delle attività intraprese dal gruppo **BitByBit** durante lo sviluppo del progetto.

Il **Piano di progetto^G** ha l'obiettivo di tenere sotto controllo l'avanzamento delle attività e di garantire l'efficienza e la qualità dello sviluppo, mediante la pianificazione delle attività e relative scadenze, monitorando i costi di ciascun periodo e identificando le potenziali criticità che ostacolerebbero lo sviluppo.

- **Piano di qualifica^G**

- Documento in cui viene definita la strategia adottata dal gruppo al fine di garantire la qualità del prodotto finale, sotto forma di metriche e relativi obiettivi, metodi di testing e resoconti.

- **Specifica Tecnica^G**

- Documento in cui il gruppo BitByBit fornisce la descrizione completa delle scelte tecnologiche e progettuali adottate per lo sviluppo del software. Oltre a definire l'**Architettura^G** logica e di **Deployment^G**, il documento espone i moduli del sistema, le tecnologie impiegate e i design pattern scelti, con lo scopo primario di fornire una base solida che guidi tutte le attività di codifica ([vedi sezione 2.2.4](#)).

- **Glossario**

- Documento contenente le definizioni dei termini usati nei documenti prodotti durante lo svolgimento del progetto, per permetterne una facile consultazione.

2.2. Processo di sviluppo

Il **Processo^G di sviluppo** comprende le attività svolte dal team di sviluppo. Esso include le attività per l'**Analisi dei requisiti^G**, il design, lo sviluppo del prodotto software, l'integrazione dei test e l'accettazione del prodotto da rilasciare.

Il **Processo^G** di sviluppo deve seguire le pratiche del **Processo^G** di gestione dei processi ([vedi sezione 4.1](#)), stabilire l'infrastruttura secondo il **Processo^G** di infrastruttura ([vedi sezione 4.2](#)), e gestire il **Processo^G** a livello organizzativo tramite i processi di miglioramento ([vedi sezione 4.3](#)) e di formazione ([vedi sezione 4.4](#)).

2.2.1 Strumenti a supporto

Per sostenere i processi di sviluppo sono stati utilizzati i seguenti strumenti:



- **Lucidchart**: piattaforma per la modellazione UML. Consente la creazione di diagrammi UML (tra cui diagrammi dei casi d'uso, delle classi e di sequenza) direttamente in ambiente web, favorendo la collaborazione sincrona tra i membri del gruppo.
- **draw.io**: strumento open source per la creazione di diagrammi UML, utilizzabile sia in ambiente web che desktop. Integra funzionalità di intelligenza artificiale che velocizzano la produzione della documentazione grafica a partire da descrizioni testuali.
- **Flutter: Framework^G** utilizzato per lo sviluppo dell'interfaccia **Utente^G** dell'applicazione mobile.
- **Visual Studio Code: IDE^G** adottato dal gruppo come ambiente di sviluppo comune, configurato con le estensioni necessarie per supportare la formattazione automatica del codice Dart e Python ([vedi Sezione 2.2.4.1](#), [vedi Sezione 2.2.4.2](#)).
- **Git**: sistema di controllo di versione distribuito utilizzato per tracciare le modifiche al codice sorgente e coordinare il lavoro tra i membri del gruppo.
- **GitHub**: piattaforma di hosting per **Repository^G** Git, utilizzata per la gestione del codice sorgente, il tracciamento delle **Issue^G** e la revisione del codice tramite **Pull Request^G**.
- **GitHub Actions^G**: piattaforma di **CI/CD^G** utilizzata per automatizzare i controlli di qualità ad ogni *push*. Esegue la formattazione automatica del codice, l'**Analisi Statica^G** tramite SonarQube e blocca il **Merge^G** nel ramo principale in caso di esito negativo.
- **AWS SAM: Framework^G** open source utilizzato per la build e il deploy dei servizi AWS. Consente di definire l'infrastruttura **Serverless^G** tramite file di configurazione e di automatizzarne il rilascio tramite la pipeline di **CI/CD^G**.

2.2.2 Attività previste

Le attività che compongono il **Processo^G di sviluppo** adattate al progetto sviluppato dal gruppo BitByBit sono elencate di seguito.

- **Implementazione di Processo^G**, l'organizzazione è tenuta a scegliere un modello di **Ciclo di vita^G** se non ne ha ancora adottato uno in particolare, tenendo conto del campo di applicazione, del campo e della complessità del progetto.
- **Analisi dei requisiti^G di sistema**, l'analisi del sistema da implementare deve essere condotta in modo dettagliato, al fine di produrre una descrizione delle funzionalità e delle capacità del sistema, nonché dei requisiti **Utente^G** e di sistema.



- **Progettazione dell'Architettura^G**, deve essere identificata un'Architettura^G di alto livello per identificare gli elementi software. Deve essere garantito che tutti i requisiti siano allocati tra gli elementi.
- **Codifica e testing**, per ogni elemento software il programmatore deve implementare e documentare ogni elemento software, andando a garantire che ogni unità software soddisfi i suoi requisiti implementando i corrispettivi test.
- **Integrazione software**, il programmatore è tenuto a definire le unità software, i test le tempistiche e i dati che devono essere implementati per il particolare elemento software.
- **Test di qualità del software**, comprende l'implementazione di test in conformità con le metriche di qualità del software.
- **Integrazione nel sistema**, il programmatore procede ad integrare l'elemento software nel sistema, assieme alle sue dipendenze se è necessario.
- **Test di qualità del sistema**, vengono eseguiti i test sull'intero sistema, per accertarsi che sia pronto per essere rilasciato.
- **Installazione del software**, comprende la consegna del software al cliente e la sua installazione nell'ambiente concordato dal **Contratto^G**.
- **Supporto per l'approvazione**, il fornitore e il cliente collaudano il prodotto per accertare che tutti i requisiti stabiliti sono presenti nel software consegnato.

2.2.3 Analisi Dei Requisiti

L'attività di **Analisi dei requisiti^G** ha l'obiettivo di trasformare i requisiti **Utente^G** espressi nel **Capitolato^G**, nel nostro caso il **Capitolato^G C4** proposto da **Miriade S.r.l.**, in requisiti di sistema chiari, completi e verificabili. Tale attività produce il documento dell'**Analisi dei requisiti^G** che stabilisce le basi per un accordo tra la **Proponente^G** e il fornitore sulle funzionalità che verranno implementate nel prodotto.. L'**Analisi dei requisiti^G** obbliga tutte le parti coinvolte a valutare in modo completo e condiviso ciò che il sistema dovrà realizzare prima dell'avvio della progettazione. Questo riduce significativamente il rischio di dover intervenire successivamente su design, codice o test. Il gruppo si impegna inoltre a monitorare con continuità l'allineamento tra requisiti implementati e requisiti concordati con la **Proponente^G**, aggiornandoli in modo controllato quando necessario.

2.2.3.1 Stesura dei casi d'uso La definizione dei casi d'uso segue un **Processo^G** iterativo che comprende:

- **comprensione del problema** e del dominio applicativo;



- **identificazione degli scenari** attraverso la definizione delle interazioni tra attori e sistema;
- **stesura dei casi d'uso** andando ad identificare gli scenari principali e quelli secondari;
- **implementazione degli UML.**

Il gruppo effettua una revisione completa dei casi d'uso individuati, correggendo eventuali incoerenze. Se necessario, vengono condotti approfondimenti con la **Proponente^G** per garantire un'interpretazione condivisa degli scenari critici. Per ogni caso d'uso viene realizzato un diagramma UML dedicato, al fine di ridurre possibili ambiguità interpretative.

La nomenclatura adottata viene esposta di seguito.

UC-X.Y

- UC sigla per use case
- X numero intero identificativo per il caso d'uso principale;
- Y numero intero identificativo per il sotto-caso eventuale.

Questa codifica garantisce un'identificazione univoca e promuove tracciabilità e verificabilità.

2.2.3.2 Stesura dei requisiti I requisiti vengono organizzati per livello di importanza, definito con la **Proponente^G**:

- **Obbligatorî (OB):** necessari e irrinunciabili per la soddisfazione minima degli **Stakeholder^G**;
- **Desiderabili (DE):** non essenziali ma ad alto valore aggiunto;
- **Opzionali (OP):** utili, ma pianificabili o negoziabili in fasi successive.

Ogni **Requisito^G** è identificato tramite una codifica standard:

R<tipo>-<importanza>-X

Dove:

- R: è per abbreviare la parola **Requisito^G**
- <tipo>: è la sigla che indica la tipologia di **Requisito^G**
 - F: per indicare che è un **Requisito funzionale^G**
 - Q: per indicare che è un **Requisito^G** di Qualità



- V: per indicare che è un **Requisito^G** di Vincolo
- <importanza>: è la sigla a due lettere che indica la categoria del **Requisito^G**
- X: è un identificatore numerico univoco

Tale classificazione supporta la tracciabilità, la **Verifica^G** e la gestione dell'evoluzione dei requisiti durante l'intero **Ciclo di vita^G**.

2.2.4 Codifica

L'attività di codifica ha lo scopo di realizzare tecnicamente quanto definito nelle fasi di analisi e progettazione. L'obiettivo principale delle norme qui esposte è garantire l'**uniformità**: il codice deve risultare omogeneo, leggibile e manutenibile, indipendentemente dal membro del gruppo che lo ha scritto.

2.2.4.1 Stile di Codifica: Dart

2.2.4.1.1 Convenzioni di nomenclatura Dart Il gruppo adotta le convenzioni ufficiali definite da *Dart* per garantire uniformità e leggibilità del codice:

1. **Nomi di classi, enum, typedef ed estensioni**: si utilizza **UpperCamelCase^G** (es. `UserRepository`, `AuthService`).
2. **Nomi di variabili, parametri, metodi e costanti**: si utilizza **lowerCamelCase^G** (es. `userName`, `fetchData`).
3. **Nomi di file, directory e prefissi di import**: si utilizza **lowercase_with_underscores** (es. `user_repository.dart`, `auth_service.dart`).
4. **Lingua del codice**: nomi di classi, variabili, metodi e file devono essere in **lingua inglese**.
5. **Lingua dei commenti**: i commenti nel codice devono essere redatti in **lingua italiana**.
6. **Nomenclatura file**: la nomenclatura di ogni file deve corrispondere al nome della classe al suo interno.
7. **Commenti di documentazione**: ogni metodo deve essere documentato tramite un commento (usando il comando `///`) che ne descrive il suo scopo. I riferimenti a variabili o parametri all'interno dei commenti vanno racchiusi tra parentesi quadre (es. `/// Restituisce il valore di [userId]`).
8. **File di test**: i file contenenti test unitari devono terminare con il suffisso `_test.dart` (es. `user_repository_test.dart`).



2.2.4.1.2 Convenzioni nomenclatura delle classi Dart Al fine di rendere immediatamente riconoscibile il ruolo architetturale di una classe dalla sua dichiarazione, il gruppo adotta i seguenti suffissi obbligatori:

- **ViewModel**: classi della componente view model del **UI Layer^G** per la gestione dello stato e della logica di presentazione di una schermata (es. `LoginViewModel`).
- **Screen**: classi widget Flutter del **UI Layer^G** che rappresentano una schermata completa (es. `LoginScreen`).
- **Widget**: classi che rappresentano un widget Flutter del **UI Layer^G** non associato a una singola schermata (es. `AvatarWidget`).
- **Repository**: classi del **Data Layer^G** responsabili della Persistence Logic (es. `UserRepository`).
- **Service**: classi del **Data Layer^G** responsabili della **Business Logic^G** (es. `AuthService`).

2.2.4.1.3 Formattazione automatica del codice Dart La formattazione del codice Dart viene delegata al comando nativo `dart format`, che applica automaticamente le regole stilistiche definite da *Dart*. Per garantirne il rispetto su tutto il progetto, la formattazione automatica viene integrata su due livelli per un maggiore controllo sul codice che i membri del gruppo producono durante questa attività:

- **Locale**: l'**IDE^G** adottato dal gruppo è configurato per eseguire il comando `dart format` al salvataggio del file.
- **Remoto**: la pipeline di **GitHub Actions^G** per eseguire la Ci applica la formattazione Dart ad ogni *push*, impedendo il **Merge^G** di codice non formattato correttamente nel ramo principale.

2.2.4.1.4 Convenzioni sulla struttura gerarchica delle directory La struttura del codice del progetto Flutter segue la suddivisione per layer raccomandata dalla documentazione ufficiale di Flutter. Di seguito viene riportata la struttura delle directory principali e il contenuto atteso per ciascuna:

- **lib/**, directory del codice sorgente dell'applicazione:
 - **ui/**, contiene tutto il codice relativo alla presentazione:
 - * **core/**, contiene i widget riutilizzabili condivisi da più schermate (classi con suffisso `Widget`);
 - * **<feature_name>/**, ogni funzionalità ha una directory dedicata, al suo interno:



- `view_model/`, contiene le classi `ViewModel` associate alle schermate della funzionalità;
- `widget/`, contiene le classi `Screen` e gli eventuali widget specifici della funzionalità.
- `domain/`, contiene le classi che rappresentano i dati nel formato consumato dal layer UI, indipendenti da qualsiasi sorgente dati;
- `data/`, contiene il layer di accesso ai dati, suddiviso in:
 - * `dtos/`, contiene le classi `DTO` (Data Transfer Object), che rappresentano la struttura dei dati così come vengono inviati alla sorgente dati, prima di eventuali trasformazioni;
 - * `proxies/`, contiene le classi `Proxy`, che rappresentano implementazioni fittizie dei `Service`, utilizzate principalmente per i test;
 - * `repositories/`, contiene le classi **`RepositoryG`**, che rappresentano la fonte di verità per un tipo di dato e orchestrano l'accesso ai `Service`;
 - * `services/`, contiene le classi `Service`, che comunicano con sorgenti dati esterne (**`API RESTG`**, database locali, ecc.).
- `utils/`, contiene classi e funzioni di utilità condivise tra più componenti del progetto, come ad esempio helper per la gestione degli errori, formattazione delle date, ecc.
- `test/`, contiene gli unit test del codice Dart. La struttura interna rispecchia quella di `lib/`, in modo che ogni file sorgente abbia il corrispondente file di test con suffisso `_test.dart`;
- `testing/`, contiene le classi dedicate ai mock e alle utilities condivise tra i test, come le implementazioni fittizie dei **`RepositoryG`** e dei `Service`.

2.2.4.2 Stile di Codifica: Python

2.2.4.2.1 Convenzioni di nomenclatura Python Il gruppo adotta le convenzioni ufficiali definite da *PEP 8* per garantire uniformità e leggibilità del codice Python utilizzato nelle funzioni Lambda:

1. **Classi:** si utilizza **`UpperCamelCaseG`** (es. `UserHandler`, `ApiResponse`).
2. **Funzioni, variabili e parametri:** si utilizza `lowercase_with_underscores` (es. `user_id`, `get_user`).
3. **Costanti:** si utilizza `UPPER_CASE_WITH_UNDERSCORES` (es. `MAX_RETRY_COUNT`, `BASE_URL`).
4. **File e moduli:** si utilizza `lowercase_with_underscores` (es. `user_handler.py`, `api_utils.py`).



5. **Lingua codice:** nomi di classi, variabili, funzioni e file devono essere in **lingua inglese**.
6. **Lingua commenti:** i commenti nel codice devono essere redatti in **lingua italiana**.
7. **Commenti di documentazione:** ogni funzione deve essere documentata tramite una *docstring* (usando le triple virgolette `"""`) che ne descrive lo scopo, i parametri e il valore di ritorno.
8. **File di test:** i file contenenti test unitari devono iniziare con il prefisso `test_` (es. `test_user_handler.py`).

2.2.4.2.2 Convenzioni sui suffissi delle classi Python Al fine di rendere immediatamente riconoscibile il ruolo di una classe dalla sua dichiarazione, il gruppo adotta i seguenti suffissi obbligatori:

- **Handler:** classi che gestiscono l'evento in ingresso alla funzione Lambda e orchestrano la risposta (es. `UserHandler`).
- **Service:** classi che incapsulano la **Business Logic^G**, indipendentemente dal tipo di evento ricevuto (es. `NotificationService`).
- **Repository:** classi che gestiscono l'accesso ai dati, ad esempio verso DynamoDB o RDS (es. `UserRepository`).
- **Client:** classi che incapsulano la comunicazione con servizi AWS esterni alla Lambda, come S3 o SNS (es. `S3Client`).

2.2.4.2.3 Formattazione automatica del codice Python La formattazione del codice Python viene delegata al tool **black**, che applica automaticamente le regole stilistiche definite da *PEP 8*. Per garantirne il rispetto su tutto il progetto, la formattazione automatica viene integrata su due livelli:

- **Locale:** l'**IDE^G** adottato dal gruppo è configurato per eseguire **black** al salvataggio del file tramite l'estensione ufficiale.
- **Remoto:** la pipeline di **GitHub Actions^G** per eseguire la Ci applica la formattazione Python ad ogni *push*, impedendo il **Merge^G** di codice non formattato correttamente nel **Branch^G** principale.

2.2.4.3 Vincoli implementativi Al fine di mantenere il codice robusto e testabile, il gruppo adotta le seguenti pratiche ingegneristiche:

- **Assenza di stato globale:** è vietato l'utilizzo di variabili globali, in quanto introducono dipendenze nascoste e rendono imprevedibile il comportamento del software.



- **Semplicità delle funzioni:** le funzioni devono essere brevi e focalizzate su una singola responsabilità. Una funzione eccessivamente lunga o complessa è indice di cattiva progettazione e deve essere rifattorizzata.
- **Immutabilità:** si preferisce l'uso di strutture dati immutabili e variabili dichiarate `final` (Dart) o tipizzate come `Final` (Python) ove possibile, riducendo il rischio di modifiche accidentali allo stato.

3. Processi di supporto

La presente sezione espone i seguenti processi di supporto:

- (a) **documentazione;**
- (b) **gestione della configurazione;**
- (c) **gestione della qualità;**
- (d) **Verifica^G;**
- (e) **Validazione^G.**

3.1. Documentazione

Il **Processo^G** di documentazione ha l'obiettivo di definire le modalità, gli strumenti e le convenzioni che il gruppo adotta per la produzione, la gestione, la revisione e l'approvazione di tutti i documenti del progetto.

3.1.1 Obiettivi

1. Garantire **completezza, chiarezza e coerenza** delle informazioni contenute nei documenti.
2. Assicurare la **tracciabilità** delle modifiche e delle versioni;
3. Favorire la **comprensibilità e manutenibilità** dei documenti da parte di tutti i membri del gruppo;
4. Fornire un **mezzo ufficiale di comunicazione** tra il gruppo, il **Committente^G** e i revisori;
5. Rispettare la **uniformità formale e stilistica** tra i vari documenti prodotti.

L'implementazione del **Repository^G** dedicato alla documentazione integra tutti gli strumenti necessari alla produzione dei documenti di **Ciclo di vita^G**, garantendo che ogni nuovo membro del progetto possa svolgere le attività previste senza dipendere dal proprio



ambiente di lavoro locale o incorrere in limitazioni tecniche.

È necessaria l'automazione dei processi dove è possibile, in modo da avere un sistema con pipeline per lo sviluppo che aiuti il membro del gruppo nei suoi compiti.

3.1.2 Strumenti a supporto

Per sostenere il **Processo^G** di documentazione sono utilizzati i seguenti strumenti:

- **L^AT_EX**, per la stesura di documenti il gruppo ha scelto L^AT_EX, linguaggio di markup che permette di avere il controllo di tutte le parti di un documento, può essere eseguito nel **Repository^G** senza la necessità di un editor di testo;
- **Overleaf**, piattaforma online utilizzata in fase iniziale per l'apprendimento della sintassi e i comandi di L^AT_EX;
- **Google Sheets**: utilizzato per la redazione di pianificazioni, tracciamento di requisiti preliminari, matrici decisionali e supporto alla raccolta dati;
- **Google Docs**, impiegato durante riunioni o fasi preliminari per appunti condivisi e raccolta rapida di contenuti;
- **Google Presentazioni**: utilizzato per la redazione e condivisione dei diari di bordo richiesti dal corso;
- **GitHub**, piattaforma di versionamento distribuito e piattaforma di collaborazione utilizzata per la gestione centralizzata dei documenti, delle modifiche, dei **Branch^G** e delle **Pull Request^G**;
- **GitHub Actions^G**, strumento di automazione per l'aggiunta del template comune a tutti i documenti e per la compilazione dei file `.tex` in PDF.

3.1.3 Tipologie di documenti

- **Documenti interni**, destinati alla gestione e al coordinamento interno (verbali, norme di progetto, glossario);
- **Documenti esterni**, destinati a revisori, docenti o aziende (**Analisi dei requisiti^G**, **Piano di progetto^G**, **Piano di qualifica^G**);
- **Verbali interni ed esterni**, registrano rispettivamente, gli incontri tra i membri del gruppo e gli incontri tra il gruppo e gli enti esterni come l'azienda **Proponente^G** o i professori.

Ogni documento, indipendentemente dalla sua categoria, deve essere redatto in conformità alle convenzioni formali definite nelle Norme di Progetto.



3.1.4 Implementazione di Processo

Ogni documento, prima di essere integrato nella versione stabile del **Repository^G**, deve attraversare un ciclo strutturato di produzione che garantisce la qualità, la verificabilità e la tracciabilità delle modifiche. Le attività descritte in questa sezione definiscono il flusso operativo seguito dal gruppo per l'elaborazione, la revisione e la pubblicazione dei prodotti documentali.

3.1.4.1 Creazione delle Issue GitHub Ogni azione che porta a dei cambiamenti ad un documento del gruppo deve iniziare con l'apertura di una **Issue^G** su GitHub per permettere la tracciabilità nella board del GitHub **Project^G**. Ogni commit deve fare riferimento ad una sola azione in modo da garantire che il **Processo^G** iterativo di revisione ed eventuali correzioni di errori sia il più veloce possibile.

3.1.4.2 Assegnazione della Issue Nessuna **Issue^G** può essere avviata senza che sia assegnata ad un membro del gruppo. L'assegnazione viene effettuata dal responsabile, che determina chi, nel gruppo, è incaricato dell'esecuzione di quella precisa attività. Questo passaggio garantisce che ogni modifica sia riconducibile a un autore e che la responsabilità operativa sia chiaramente definita.

3.1.4.3 Impostazione del documento Il membro del team può ora aprire un **Branch^G** secondario da **develop**, seguendo la nomenclatura presente nella sezione [4.2.3](#), ed eseguire le modifiche definite nella relativa **Issue^G**. Se le modifiche implicano la creazione del file di documentazione il membro del gruppo è tenuto a creare il file sorgente, denominato secondo le convenzioni stabilite ([vedi sezione 4.2.3](#)). Una volta inserito il nuovo file nel **Repository^G** remoto, il sistema provvede automaticamente ad applicare il template comune, garantendo l'integrazione delle componenti strutturali condivise.

Spetta quindi al redattore adattare il template al contenuto specifico del documento, completandone le sezioni pertinenti e verificando che l'impostazione risultante sia conforme agli standard definiti per la documentazione del progetto. Questa attività assicura uniformità, leggibilità e coerenza stilistica dell'intera produzione documentale lungo tutto il **Ciclo di vita^G** del progetto.

Ogni membro del gruppo è quindi tenuto a verificare che i documenti prodotti rispettino un formato uniforme, verificabile e riconducibile agli standard scelti.

Questa attività garantisce la leggibilità, la coerenza e la mantenibilità dell'intera produzione documentale durante tutto il **Ciclo di vita^G** del progetto.

Intestazione

Ogni pagina del documento include un'intestazione che riporta il titolo dello specifico documento e il nome del gruppo.



Frontespizio

Il frontespizio possiede i seguenti elementi:

- **logo dell'ateneo;**
- **informazioni dell'ateneo;**
- **logo del gruppo;**
- **contatto ufficiale** del gruppo;
- **nome del documento;**
- **informazioni generali del documento** le cui informazioni sono riportate di seguito.
 - **Lo stato del documento;**
 - la **versione** attuale;
 - il membro del gruppo **responsabile** per quel documento;
 - **redattori**: i membri del gruppo che hanno contribuito alla redazione
 - i **verificatori**, che identificano i membri che hanno eseguito attività di **Verifica^G** per quel documento;
 - i **destinatari** del documento.

Storico del documento

Nella prima pagina dopo il frontespizio deve essere presente il registro delle modifiche apportate al documento. Le entry della tabella devono essere in ordine cronologico in modo che la prima entry faccia riferimento alla modifica più recente.

Ogni entry deve essere composta da:

1. **versione;**
2. **data della modifica;**
3. **autore della modifica;**
4. **verificatore**, il nome del verificatore che ha verificato quella modifica.

Versione	Data	Autore	Descrizione	Verificatore

Tabella 2: Esempio di tabella dello storico di un documento



Indice

Ogni documento deve presentare un indice delle sezioni e sottosezioni per facilitare la navigazione dei lettori all'interno dello stesso documento.

Struttura dei verbali

I verbali rappresentano i rendiconti di riunioni ufficiali interne, a cui partecipano solo i componenti del gruppo, ed esterne, a cui invece partecipano anche enti esterni al gruppo come impiegati dell'azienda **Proponente^G** del **Capitolato^G** o i professori del corso. Ogni **Verbale^G** oltre alle sezioni precedentemente elencate deve presentare al suo interno i seguenti elementi nell'ordine in cui sono descritti:

1. **Informazioni generali** per quell'incontro: contiene i dati identificativi dell'incontro quali:
 - la data in cui si è svolto l'incontro;
 - l'orario d'inizio;
 - la durata;
 - il luogo in cui si è svolto;
 - l'elenco dei presenti e degli assenti.

Queste informazioni sono volte a dare un contesto formale alla riunione.

2. **Ordine del giorno**: riassume i punti principali che si intendono discutere durante l'incontro, predisposti in anticipo dal responsabile/i di progetto durante quel periodo. Questa sezione funge da guida per la conduzione del **Verbale^G**.
3. **Discussioni**: riporta in modo sintetico ma chiaro i contenuti emersi durante il confronto tra i partecipanti. Devono essere descritte le osservazioni, le problematiche sollevate e le proposte di soluzione valutate durante l'incontro.
4. **Decisioni**: elenca tutte le risposte che il gruppo si dà per uno specifico punto, incluse quelle riguardanti modifiche ai documenti, scadenze o variazioni organizzative. Ogni decisione deve essere formulata in modo oggettivo e deve avere un codice identificativo univoco che segue il seguente formato:

`<VI/VE> <fase a cui appartiene> Y.Z`

dove:

- **VI/VE**, indica se la decisione fa riferimento ad un **Verbale^G** interno o esterno, seguendo le convenzioni di scrittura stabilite.
- **fase**, indica la fase di progetto a cui si riferisce;



- **Y**, indica il numero del **Verbale^G** in riferimento a quella revisione;
 - **X** indica il numero della decisione interna a quel documento.
5. **To-Do^G**: presenta, sotto forma di tabella, gli incarichi operativi pianificati a seguito della riunione. Ogni **To-Do^G** è identificato da:
- una descrizione sintetica;
 - Il codice della decisione da cui è stata creata (per ogni decisione Ci possono essere zero o più attività);
 - il numero della **Issue^G** GitHub di quella attività presente nel GitHub **Project^G**.

Convenzioni di scrittura

Le convenzioni di scrittura stabiliscono regole comuni per la redazione di tutti i documenti del progetto.

3.1.4.4 Date

- Le **date** devono essere nel formato **AAAA-MM-GG**.

Le ore hanno 2 formati:

- **Durata**: Per esprimere una durata usare il formato **Hh Mm** (es: 2h 15m)
- **Ora di orologio**: Per indicare che un evento è avvenuto in una specifica ora, usare il formato **HH:MM** (es: 15:45)

Nomi dei file

- Tutti i file devono avere nomi in minuscolo, senza spazi, e le parole devono essere separate da un **underscore** (**_**).
- Il nome di ogni **Verbale^G** deve iniziare con **verbale_**, segue il tipo di **Verbale^G** **interno** o **esterno** e la data in cui si è svolto. Un esempio di nominazione corretta è **verbale_interno_2025-11-05**.
- L'unica eccezione è nel caso in cui usare questo formato generi due verbali con lo stesso nome all'interno della stessa cartella. In tal caso è necessario aggiungere, dopo la data, una breve descrizione del contesto relativo al **Verbale^G**, sostituendo agli spazi il simbolo **underscore** (**_**). Ad esempio **verbale_2025-10-28_incontro_c8**.
- I file PDF mantengono lo stesso nome del file sorgente **.tex**.



Sigle

Di seguito sono presenti le sigle usate all'interno dei documenti.

- **Fasi di progetto:**
 - **C**, Candidatura;
 - **RTB^G**, Requirements and Technology **Baseline^G**;
 - **PB^G**, Product **Baseline^G**.
- **Tipologie di documenti:**
 - **AdR**, **Analisi dei requisiti^G**;
 - **NdP**, norme di progetto;
 - **PdQ**, **Piano di qualifica^G**;
 - **PdP**, **Piano di progetto^G**;
 - **G**, glossario;
 - **VI**, **Verbale^G** interno;
 - **VE**, **Verbale^G** esterno.

Nomi dei membri

- I nomi dei membri devono essere riportati nel formato: **Nome Cognome** (iniziali maiuscole, nessuna abbreviazione).

3.1.4.5 Modifiche a documenti Per eseguire delle modifiche alla documentazione di progetto, il membro del gruppo è tenuto ad istanziare un nuovo **Branch^G** a partire da quello di **develop**. La denominazione del **Branch^G** di feature deve seguire la nomenclatura presente nella sezione ([vedi sezione 4.2.3](#)).

Ogni modifica a un documento deve essere accompagnata da uno scatto del numero di versione. Il membro del gruppo che si occupa delle modifiche è anche tenuto ad aggiornare lo **storico dei documenti** con le informazioni richieste.

Ogni modifica ad un documento deve essere tracciabile all'interno dello stesso tramite lo **storico del documento**, riportando:

- il numero di versione assegnato;
- la data della conclusione della modifica;
- il nome e cognome del membro del team che l'ha eseguita;
- una descrizione concisa, sintetica e formale delle modifiche avvenute;
- il nome del verificatore che è tenuto ad eseguire la **Verifica^G**.



3.1.4.6 Aggiornamento del glossario Ogni membro del gruppo, nel momento in cui aggiunge un termine ad un documento che può essere fonte di ambiguità e che non sia già presente nel glossario, è tenuto ad aggiornare il glossario del progetto. Con il termine deve essere anche fornita una sua descrizione esplicativa.

3.1.4.7 Conclusione modifiche Una volta completate le modifiche nel nuovo **Branch^G** aperto, il membro incaricato procede ad aprire una **Pull Request^G** dal proprio **Branch^G** di feature verso il **Branch^G develop**. Nella fase di apertura della **Pull Request^G** deve essere compilato il form per la descrizione della **Pull Request^G** con le informazioni richieste. La **Pull Request^G** costituisce la richiesta formale per l'integrazione delle modifiche nel **Repository^G** del gruppo, dopo l'avvenuta approvazione delle modifiche apportate.

3.1.4.8 Verifica Il verificatore esamina sia la correttezza formale sia sostanziale del documento.

- Se le modifiche risultano corrette, il verificatore approva la **Pull Request^G**.
- Se emergono incongruenze o problemi, il verificatore richiede delle modifiche aggiungendo dei commenti al codice.

Le modifiche richieste devono essere applicate dall'autore originario della **Pull Request^G**, che procede aggiornando il **Branch^G develop** fino alla completa risoluzione. I verificatori sono tenuti a controllare che i documenti di **Ciclo di vita^G** del progetto comprendano gli elementi di struttura elencati di seguito nell'ordine in cui si presentano.

3.1.4.9 Approvazione e Pull Request Una volta ottenuta l'approvazione del verificatore, la **Pull Request^G** viene sottoposta alla visione del responsabile, che effettua un'ultima revisione formale. Solo il responsabile è autorizzato a procedere con il **Merge^G** della **Pull Request^G** nel **Branch^G develop**.

Il **Merge^G** rappresenta l'integrazione definitiva del documento nella versione stabile del progetto.

3.1.4.10 Pubblicazione Con la conclusione del **Merge^G**, il documento aggiornato diventa parte integrante della **Baseline^G** di progetto. Il **Branch^G main** costituisce, infatti, il riferimento ufficiale per la versione stabile e coerente dei documenti.

3.2. Processo di gestione della configurazione

Il **Processo^G** di gestione della configurazione ha l'obiettivo di applicare procedure per garantire il controllo sistematico degli artefatti prodotti dal gruppo *BitByBit* durante l'intero **Ciclo di vita^G** del software.

Esso permette di tracciare le modifiche apportate ai **Configuration Item^G**, garantendo



che ogni item venga rilasciato in modo coerente e dopo le opportune verifiche. Inoltre, consente di registrare, classificare e monitorare le richieste di modifica, assicurando che ogni **Configuration Item^G** sia completo, coerente e corretto. Infine, supporta una gestione ordinata degli item approvati, curandone l'archiviazione, la conservazione e la distribuzione.

3.2.1 Strumenti a supporto

Per le attività previste, il gruppo BitByBit utilizza gli strumenti elencati di seguito.

- **GitHub**: piattaforma per il versionamento, la gestione dei **Repository^G**, delle **Issue^G**, delle **Pull Request^G** e dei flussi di lavoro collaborativi mediante GitHub Projects e Teams. È inoltre il punto ufficiale di archiviazione degli artefatti approvati.
- **GitHub Actions^G**: utilizzate per automatizzare la compilazione dei documenti in $\text{\LaTeX}$ e garantire la **Validazione^G** continua del **Repository^G**.
- **Branch^G protection rules**: utilizzate per impedire modifiche non autorizzate sul **Branch^G** principale e garantire che ogni integrazione avvenga secondo le regole di **Verifica^G** previste dal gruppo.

3.2.2 Attività previste

Le attività identificate dal gruppo per il **Processo^G** di gestione della configurazione sono:

- **identificazione della configurazione**, viene definito uno schema uniforme di versionamento e naming, per identificare ogni componente del sistema;
- **controllo della configurazione**, metodologie per stabilire l'identificazione, l'approvazione o rifiuto, la **Verifica^G** e l'eventuale rilascio delle richieste di modifica;
- **registro di stato**, per predisporre registri di gestione e report di stato che mostrino lo stato e la cronologia degli elementi software;
- **valutazione della configurazione**, come determinare la completezza funzionale e fisica delle componenti software con i loro requisiti.

3.2.3 Identificazione della configurazione

Per il versionamento viene adottato il Semantic Versioning **SemVer^G**. Ogni documento riporta una versione nel seguente formato.

X.Y.Z

Dove:



- **X (major)**: incrementata solo alla pubblicazione ufficiale, corrispondente al **Merge^G** sul **Branch^G main**;
- **Y (minor)**: incrementata quando vengono effettuate modifiche sostanziali alla struttura o al contenuto del documento;
- **Z (patch)**: incrementata con correzioni minori, refusi o chiarimenti senza impatto tecnico.

Ogni documento di **Ciclo di vita^G** presenta uno storico delle modifiche con associata la precisa versione. Questa scelta permette una chiara tracciabilità chiara, prevedibilità delle modifiche ed evoluzione controllata degli artefatti documentali.

3.2.4 Controllo della configurazione

Il controllo della configurazione definisce le modalità con cui il gruppo gestisce, valuta e approva tutte le modifiche agli elementi configurabili del progetto.

L'intero ciclo di lavoro è organizzato secondo la pratica del feature branching, secondo la quale ogni sviluppatore deve aprire un nuovo **Branch^G** per iniziare a lavorare. Quando si è terminato il lavoro allora le modifiche possono essere integrate nel **Branch^G** principale, nel nostro caso identificato su develop.

Per la nomenclatura del **Branch^G** di **feature** ogni membro del gruppo è tenuto a seguire la nomenclatura di seguito esposta.

`task/<descrizione delle modifiche>_<documento soggetto alle modifiche>`

Dove:

- `<descrizione delle modifiche>`, comprende una breve descrizione delle modifiche applicate in quel **Branch^G**, senza utilizzare spazi e le parole separate da **trattini (-)**;
- `<documento soggetto alle modifiche>`, sigla (che segue la nomenclatura in [Sezione 3.1.4.4](#)) per il documento a cui avvengono apportate le modifiche.

3.2.4.1 Gestione Issue e pianificazione del lavoro Ogni modifica nasce dall'apertura di una **Issue^G** GitHub, che costituisce il punto di riferimento per la tracciabilità dell'attività.

Al momento della creazione, la **Issue^G** deve essere dotata di un titolo descrittivo e di una spiegazione chiara dell'obiettivo. Il responsabile assegna inoltre priorità, **Milestone^G**, iterazione (che corrisponde allo **Sprint^G** in cui l'attività deve essere completata) e l'assegnatario, selezionato in base al ruolo che il membro ricopre in quel momento. Label aggiuntive consentono una categorizzazione accurata del lavoro.

La pianificazione complessiva è supportata dalle GitHub **Project^G** Board, che permettono di consultare **Product backlog^G** e **Sprint backlog^G** tramite viste personalizzate, semplificando la gestione dei carichi di lavoro e l'avanzamento delle attività.



3.2.4.2 Creazione della Pull Request Una volta completata l'attività descritta nella **Issue^G** GitHub, il membro incaricato apre una **Pull Request^G** dal **Branch^G** di feature al **branch develop**.

Ogni **Pull Request^G** deve utilizzare il template comune del gruppo, che contiene le sezioni necessarie per descrivere contesto, modifiche introdotte, **Issue^G** collegate e controlli effettuati. La descrizione del template presente nel file `pull_request_template.md`, deve essere compilato e adattato ogni volta, così da velocizzare il lavoro di revisione e rendere uniformi tutte le richieste. Il membro deve indicare tramite il campo **Reviewers** il singolo membro incaricato del compito di verificare quella modifica.

3.2.4.3 Verifica e revisione della Pull Request All'apertura della **Pull Request^G** viene richiesta automaticamente la **Verifica^G** ai membri del team verificatori:

- se la **Verifica^G** produce esito positivo, la **Pull Request^G** viene approvata dal verificatore incaricato;
- se sono presenti dei problemi o miglioramenti da applicare, il verificatore richiede delle modifiche, tramite l'opzione di GitHub `request changes`, aggiungendo un commento di quello che è il problema trovato e di una possibile soluzione se è presente. Successivamente la persona che ha aperto la **Pull Request^G** si impegna a eseguire le correzioni e a notificare il completamento dell'azione per avere una **Verifica^G** il prima possibile.

3.2.4.4 Integrazione nella Baseline Quando la **Pull Request^G** viene approvata dal verificatore, il responsabile esegue la revisione. Si impegna ad eseguire il **Merge^G** nel **Branch^G develop**. A questo punto, la modifica entra ufficialmente in **Baseline^G** e diventa parte del progetto.

3.2.5 Registro dello stato della configurazione

Lo stato aggiornato degli elementi di configurazione è tracciato tramite:

- le **Issue^G** aperte, in corso e chiuse;
- le **Milestone^G** attive e completate;
- la cronologia delle **Pull Request^G**;
- le versioni ufficiali dei documenti pubblicate sul **Branch^G main**.

GitHub Projects funge da registro organizzato delle attività, offrendo una rappresentazione visiva dello stato corrente.



3.2.6 Valutazione della configurazione

L'**attività di valutazione della configurazione** ha lo scopo di garantire che gli elementi software sviluppati siano completi, coerenti e conformi ai **requisiti approvati**.

Il gruppo BitByBit assicura tale controllo attraverso un tracciamento sistematico dei requisiti, che consente di collegare ogni modifica e ogni componente alla motivazione tecnica o funzionale che la giustifica. Durante lo sviluppo, i membri si impegnano inoltre a mantenere una chiara corrispondenza tra codice e requisiti, anche tramite annotazioni e riferimenti interni.

La valutazione viene svolta in modo continuativo, non solo nelle fasi conclusive, così da individuare rapidamente deviazioni dal design e garantire che:

- tutti i requisiti software siano effettivamente soddisfatti;
- non vengano introdotti comportamenti non richiesti;
- ogni parte del sistema risulti coerente con l'**Architettura^G** definita.

3.3. Processo di accertamento della qualità

Il **Processo^G di accertamento della qualità** ha lo scopo di garantire che la qualità del **prodotto software** e dei **processi di Ciclo di vita^G** adottati sia conforme ai requisiti specificati e ai piani prestabiliti. L'accertamento della qualità consente di fornire evidenze oggettive sul fatto che i sistemi software sviluppati siano idonei allo scopo per cui sono stati richiesti e soddisfino le aspettative degli **Stakeholder^G** coinvolti.

3.3.1 Implementazione di Processo

Il gruppo *BitByBit* ha identificato come principale attività del **Processo^G di accertamento della qualità** la redazione del documento di **Piano di qualifica^G**. Tale documento definisce gli obiettivi quantitativi di qualità relativi sia al prodotto software sia ai processi di **Ciclo di vita^G** adottati, e raccoglie le misurazioni oggettive effettuate durante lo svolgimento del progetto.

Le misurazioni consistono nella quantificazione, mediante l'utilizzo di metriche, di specifici attributi dei prodotti software e dei processi di **Ciclo di vita^G**. Per ciascuna **Metri-ca^G** vengono inoltre individuati, nel **Piano di qualifica^G**, dei valori ottimali o soglie di riferimento, al fine di identificare tempestivamente eventuali comportamenti anomali o scostamenti rispetto agli obiettivi di qualità prefissati.

Un elenco completo delle metriche adottate è riportato alla [Sezione 5](#) per una consultazione più agevole.



3.3.2 Accertamento della qualità di prodotto

L'attività di **accertamento della qualità di prodotto** ha lo scopo di assicurare che i prodotti software e la documentazione correlata siano forniti in conformità ai termini e ai requisiti specificati con l'acquirente.

Nel **Piano di qualifica^G** vengono definite specifiche metriche di qualità di prodotto. Le misurazioni eseguite vengono confrontate con i valori ottimali stabiliti per ciascuna **Metrica^G**, al fine di valutare in modo oggettivo la qualità del software realizzato.

Le metriche di prodotto costituiscono un supporto decisionale per determinare se il software implementato soddisfa i criteri di qualità richiesti ed è pertanto idoneo al rilascio.

3.3.3 Accertamento della qualità di processo

L'attività di **accertamento della qualità di Processo^G** ha lo scopo di assicurare che i processi di **Ciclo di vita^G** siano eseguiti in conformità agli standard e alle procedure definiti per ciascun **Processo^G**.

Nel **Piano di qualifica^G** vengono formalizzate le metriche di **Processo^G** e i relativi valori ottimali stabiliti dal gruppo. Le misurazioni effettuate vengono successivamente confrontate con tali valori di riferimento, consentendo di valutare l'efficacia e l'adeguatezza dei processi adottati.

Le metriche di **Processo^G** sono utilizzate dal responsabile di progetto come strumento di supporto decisionale per individuare eventuali criticità e valutare la necessità di apportare modifiche o miglioramenti ai processi in uso.

3.4. Processo di Verifica

Il **Processo^G di Verifica^G** è l'insieme delle attività volte ad accertare che ogni prodotto del **Ciclo di vita^G** (codice, documenti, **Architettura^G**) sia conforme ai requisiti e agli standard di qualità stabiliti. L'obiettivo è rispondere alla domanda «Did we build the product right?» (Abbiamo costruito il prodotto nel modo corretto?), intercettando difetti ed errori prima che questi si propaghino alle fasi successive.

3.4.1 Strumenti a supporto

Per le attività previste dal **Processo^G di Verifica^G**, il gruppo **BitByBit** ha identificato gli strumenti elencati di seguito.

- **SonarQube**: piattaforma di **Analisi Statica^G** per la rilevazione di bug, vulnerabilità di sicurezza e code smell. Fornisce una serie di metriche già consolidate e la visualizzazione tramite indicatori dei report dei **Test di Unità^G**. I risultati sono consultabili tramite dashboard web, che fornisce una visione aggregata dello stato del codice nel tempo.



- **GitHub**: tramite l'interfaccia delle **Pull Request^G** permette ai verificatori di commentare righe specifiche del documento, richiedere modifiche e approvare il lavoro solo quando la **Checklist^G** di **Verifica^G** è soddisfatta.
- **VS Code**: tramite file di configurazione versionati permette di avere degli strumenti comuni per l'individuazione di errori di sintassi nei documenti e per l'allineamento del codice alla nomenclatura (vedi Sezione 2.2.4).

3.4.2 Attività previste

Le attività identificate dal gruppo per il **Processo^G** di **Verifica^G** sono le seguenti.

- **Implementazione di Processo^G**, si stabilisce il grado di **Verifica^G** e lo sforzo che deve essere svolto affinché questo possa essere raggiunto. Devono essere identificate le parti del sistema che possono causare le maggiori problematiche in modo che possano essere soggette ad una **Verifica^G** più approfondita tenendo conto anche delle risorse disponibili.
- **Attività di Verifica^G**, insieme di verifiche condotte sistematicamente durante il **Ciclo di vita^G** del progetto per accertare che i prodotti di ogni fase siano corretti, completi e coerenti con i requisiti. Le verifiche riguardano il **Contratto^G**, i processi, i requisiti, la progettazione, il codice, l'integrazione e la documentazione.

3.4.3 Implementazione di processo

Il gruppo *BitByBit* integra la **Verifica^G** direttamente nel flusso di sviluppo quotidiano (**Git Flow^G**), adottando la politica del **Continuous Verification^G**. Le regole operative sono le seguenti:

- **Policy "No Verify, No Merge^G"**: nessun artefatto può essere integrato nel ramo di sviluppo principale (**develop**) se non ha superato con successo i controlli di **Verifica^G**.
- **Pull Request^G come Quality Gate^G**: ogni modifica deve essere proposta tramite **Pull Request^G**. In questa sede avviene la **Verifica^G** (automatica o manuale) e solo l'esito positivo autorizza il **Merge^G**.
- **Rami Stabili**: il ramo **main** è protetto e accoglie solo versioni del software che hanno superato l'intero ciclo di test (inclusi quelli di sistema).

3.4.4 Verifica documentale

La documentazione è soggetta esclusivamente ad **Analisi Statica^G**, ovvero controlli che non richiedono l'esecuzione. Per garantire la qualità dei documenti, il gruppo valuta due metodologie:



- **Walkthrough^G**: revisione guidata dall'autore. Utile per l'allineamento del team ma dispendiosa.
- **Ispezione^G (Inspection)**: lettura autonoma da parte del verificatore basata su liste di controllo (**Checklist^G**).

Tra le due, il gruppo *BitByBit* adotta come standard l'**Ispezione^G**. L'uso di **Checklist^G** predefinite garantisce infatti un **Processo^G** ripetibile, oggettivo e meno soggetto a errori umani rispetto alla revisione libera.

3.4.5 Verifica Software

La **Verifica^G** del software si articola in due momenti distinti: l'analisi del codice sorgente (statica) e l'esecuzione dei test (dinamica).

3.4.5.1 Analisi Statica Prima di essere eseguito, il codice viene analizzato da strumenti automatici integrati nella pipeline di **CI/CD^G**. Questo passaggio serve a rilevare:

- violazioni delle norme di codifica ([vedi Sezione 2.2.4](#));
- potenziali bug logici o di sicurezza (es. variabili non utilizzate, accessi non controllati a risorse);
- code smell e violazioni dei principi di progettazione;
- il valore delle metriche di qualità definite nel **Piano di qualifica^G**, ottenendo un riscontro immediato prima che gli errori vengano introdotti nella **Code Base^G**.

L'**Analisi Statica^G** viene eseguita su ogni *push* nella pipeline di **CI/CD^G**: un esito negativo blocca il **Merge^G** nel ramo principale, garantendo che solo codice conforme agli standard definiti venga integrato. A supporto di questa attività il gruppo ha adottato **SonarQube**, scelto per la sua capacità di aggregare in un'unica dashboard i risultati dell'**Analisi Statica^G** sull'intera **Code Base^G** del sistema sviluppato.

SonarQube viene eseguito nella pipeline di **CI/CD^G** tramite **GitHub Actions^G**.

3.4.5.2 Analisi Dinamica (Testing) L'**Analisi Dinamica^G** **Verifica^G** il comportamento del software tramite la sua esecuzione. Per essere valido, ogni test deve garantire due proprietà: **ripetibilità** (stesso output a parità di input) e **automatizzabilità** (esecuzione senza intervento umano). Il gruppo struttura i test secondo il modello a V, coprendo i seguenti livelli:

1. **Test di Unità^G**: **Verifica^G** delle singole componenti (funzioni/classi) in isolamento, tramite `flutter_test` per il codice Dart e `pytest` per il codice Python.
 - **White-box^G**: test strutturali che mirano alla copertura dei percorsi logici interni (**Statement Coverage^G**).



- **Black-box^G**: test funzionali che verificano l'output dato un input, analizzando valori validi, non validi e casi limite (**Boundary Analysis^G**).
- 2. **Test di Integrazione^G: Verifica^G** della collaborazione tra unità già testate.
- 3. **Test di Integrazione^G: Verifica^G** della collaborazione tra unità già testate.
 - *Top-Down*: integrazione dalle componenti di alto livello a quelle basse (richiede **Stub^G**).
 - *Bottom-Up*: integrazione dalle componenti base verso l'interfaccia (richiede **Driver^G**).
- 4. **Test di Sistema^G: Verifica^G** del sistema completo rispetto ai requisiti funzionali e prestazionali mappati nell'**Analisi dei requisiti^G**.
- 5. **Test di Regressione^G**: riesecuzione automatica della suite di test ad ogni *push* tramite **GitHub Actions^G**, per garantire che le nuove modifiche non compromettano funzionalità preesistenti. I risultati di copertura vengono raccolti da SonarQube e sono consultabili tramite la sua dashboard.
- 6. **Test di Accettazione^G**: verifiche formali condotte con la **Proponente^G** per confermare che il software soddisfi i requisiti concordati prima del rilascio.

3.4.6 Classificazione dei Test

Per garantire l'ordine nel **Piano di qualifica^G**, il gruppo adotta una nomenclatura univoca per il tracciamento:

T-[ID]-[Tipo]

Dove **ID** è un progressivo numerico e **Tipo** indica la categoria: **U** (Unità), **I** (Integrazione), **S** (Sistema), **A** (Accettazione). Lo stato di ogni test viene tracciato come: **I** (Implementato), **NI** (Non Implementato) o **S** (Superato).

3.5. Processo di Validazione

Il **Processo^G di Validazione^G** costituisce l'ultimo livello di controllo qualitativo prima del rilascio formale del prodotto. A differenza della **Verifica^G**, che si configura come un controllo tecnico continuo sul codice, la **Validazione^G** ha lo scopo di accertare che il sistema software realizzato dal gruppo *BitByBit* sia pienamente conforme alle aspettative e ai bisogni reali dell'azienda **Proponente^G Miriade S.r.l.**



3.5.1 Obiettivi del Processo

Il gruppo adotta questo **Processo^G** per rispondere in modo affermativo alla domanda «Did we build the right product?», ovvero «Abbiamo costruito il prodotto giusto?». Gli obiettivi perseguiti sono:

- dimostrare la totale copertura dei requisiti concordati nel documento di **Analisi dei requisiti^G**;
- accertare l'idoneità del software nel contesto operativo reale tramite la simulazione di **Scenari d'uso^G**;
- ottenere l'approvazione formale per il rilascio del prodotto.

3.5.2 Tracciamento come strumento di Validazione

La prima fase operativa del **Processo^G** consiste nell'assicurare che ogni funzionalità richiesta sia effettivamente presente nel sistema. Per fare ciò il gruppo utilizza la **Matrice di Tracciamento^G** come strumento di controllo principale. L'attività prevede di mappare univocamente ogni **Requisito^G** (funzionale, di qualità o vincolo) ai relativi test di **Validazione^G**. Questo approccio metodologico garantisce che non venga sviluppato codice non richiesto e, soprattutto, che ogni **Requisito^G** concordato con **Miriade S.r.l** sia verificabile tramite un test specifico. Un **Requisito^G** privo di tracciamento verso un test è considerato non validabile e quindi non accettabile.

3.5.3 Esecuzione dei Test Di Accettazione

La **Validazione^G** si concretizza con l'esecuzione dei **Test di Accettazione^G** (UAT - User Acceptance Testing). In questa fase il gruppo non si limita a verificare l'assenza di errori, ma simula scenari di utilizzo reali (User Scenarios) per valutare il comportamento del sistema dal punto di vista dell'**Utente^G** finale. Il superamento di questi test certifica che il prodotto supporta efficacemente i processi lavorativi richiesti. In caso di fallimento di un **Test di Accettazione^G** il prodotto viene respinto e rimandato alla fase di sviluppo per le necessarie correzioni, seguite da una **Regressione^G** per confermare la risoluzione delle non conformità.

3.5.4 Strumenti a supporto della Validazione

Analogamente alla **Verifica^G**, anche il **Processo^G** di **Validazione^G** si avvale di strumenti specifici per gestire la complessità del tracciamento e l'esecuzione dei test.

3.5.4.1 Strumenti per il tracciamento La gestione manuale della **Matrice di Tracciamento^G** è soggetta ad errori umani. Per questo motivo il gruppo utilizza:



- **Script di automazione:** verranno impiegati script interni (o tool esterni integrati) capaci di scansionare i file sorgente della documentazione e del codice per generare e aggiornare automaticamente le tabelle di tracciamento (Requisiti-Fonti e Requisiti-Test).

3.5.4.2 Strumenti per l'esecuzione e il reporting

Per l'effettuazione pratica dei **Test di Accettazione**^G:

- **Ambiente di Deployment**^G: i test verranno eseguiti su un'istanza del software deployata in un ambiente il più possibile simile a quello di produzione, accessibile tramite Browser Web standard (Chrome/Firefox).
- **GitHub Issues:** qualsiasi anomalia o comportamento inatteso riscontrato durante la **Validazione**^G (test fallito) verrà tracciato aprendo una **Issue**^G dedicata su GitHub, etichettata specificamente come bug di **Validazione**^G per distinguerla dai difetti tecnici di sviluppo.

4. Processi organizzativi del Ciclo Di Vita

I processi organizzativi di **Ciclo di vita**^G rappresentano l'insieme delle attività che il gruppo BitByBit ha scelto di adottare per strutturare e supportare in modo sistematico tutti i propri progetti.

Questi processi forniscono un quadro stabile entro cui i progetti vengono sviluppati e permettono di mantenere strumenti, procedure e competenze costantemente aggiornati. Allo stesso tempo, l'esperienza maturata nei progetti può portare all'introduzione di miglioramenti nei processi stessi, contribuendo così all'evoluzione dell'intera organizzazione.

Nelle presenti *Norme di Progetto*, i processi organizzativi sono suddivisi nelle seguenti categorie, che verranno approfondite nelle sezioni successive:

- **Gestione di Processo**^G;
- **Infrastruttura**;
- **Processo**^G **di miglioramento**;
- **Processo**^G **di formazione**.

4.1. Gestione dei processi

Il **Processo**^G **di gestione dei processi** definisce le attività organizzative necessarie per pianificare, coordinare e monitorare il lavoro del gruppo. Esso stabilisce i compiti da svolgere, i ruoli incaricati di eseguirli e le modalità di comunicazione interna ed esterna. L'obiettivo è garantire che le attività procedano in modo efficace e controllato.



4.1.1 Strumenti a supporto

Il gruppo *BitByBit* utilizza le funzionalità delle **GitHub Organizations** per organizzare il lavoro e gestire i ruoli ([vedi Sezione 4.1.4](#)). Ogni team corrisponde a un ruolo specifico e include solo i membri incaricati in quel periodo, facilitando la rotazione programmata dei ruoli.

Sul **Branch^G** principale (**main**) solo i membri del team dei Responsabili possono eseguire push o **Merge^G**, mentre tutti gli altri membri propongono modifiche tramite **Pull Request^G**. La funzione **CODEOWNERS** assegna automaticamente ai Verificatori la responsabilità di revisione dei documenti, garantendo che tutte le modifiche rispettino il workflow e i controlli qualitativi previsti.

Questa configurazione assicura una chiara separazione dei ruoli, un flusso di modifica strutturato e un efficace supporto automatizzato alla gestione della configurazione.

4.1.2 Implementazione di Processo

Le attività previste dal **Processo^G** di gestione dei processi sono articolate come segue.

- **Inizializzazione e definizione degli ambiti**, il responsabile del progetto assieme ai membri del team identifica i requisiti per l'attività da svolgere. Si procede poi alla valutazione della fattibilità verificando disponibilità delle risorse, adeguatezza delle competenze e tempistiche.
- **Pianificazione**, si definisce il piano operativo del **Processo^G**, includendo la descrizione delle attività, i prodotti software da generare e la stima dei tempi necessari. Il piano specifica inoltre risorse richieste, assegnazione delle responsabilità, analisi dei rischi, misure di qualità da adottare, costi stimati e infrastruttura di supporto.
- **Esecuzione e controllo**, le attività pianificate vengono attuate. Il responsabile di **Processo^G** produce report periodici sullo stato di avanzamento, destinati sia all'uso interno sia all'acquirente. Il responsabile monitora l'aderenza al piano e interviene per risolvere eventuali scostamenti.
- **Revisione e valutazione**, perché un prodotto entri in **Baseline^G**, le modifiche apportate devono essere prima verificate da un verificatore ([si veda Sezione 3.4](#)) e successivamente revisionate dal responsabile. Quest'ultimo accerta che i prodotti software e i piani risultino conformi ai requisiti stabiliti, valutando l'esito dei compiti completati e garantendo qualità e coerenza con gli obiettivi del **Processo^G**.
- **Chiusura**, al completamento delle attività e dei prodotti previsti, si **Verifica^G** che i criteri concordati con l'acquirente siano soddisfatti. Il responsabile controlla la completezza dei risultati e dei report.



4.1.3 Ruoli

Ruoli	Compiti e responsabilità
Responsabile	È la figura che nel momento governa il team. Identifica le attività che sono state eseguite nello Sprint^G terminato e quelle che non sono state portate a termine. Durante l'evento di Sprint^G Planning stabilisce ed inserisce nello Sprint backlog^G le attività da eseguire nello Sprint^G successivo, individuando i costi, i rischi e i membri del gruppo in base ai loro ruoli. Rappresenta il gruppo all'esterno, interfacciandosi con gli enti esterni quali i professori e l'azienda Proponente^G .
Amministratore	È l'amministratore di sistema (Sys Admin), si occupa della gestione dell'infrastruttura utilizzata per eseguire le attività dei processi di Ciclo di vita^G . Ha il compito di inserire nell'infrastruttura gli strumenti e le tecnologie per l'attuazione del Way of working^G . Deve accertarsi che l'infrastruttura si trovi in uno stato stabile e quindi risolvere eventuali problematiche che possono derivarne. Deve redigere il presente documento.
Analista	Il compito dell'analista è quello di identificare i requisiti del progetto, interpretando le necessità degli utenti finali in modo da ottenere una corretta definizione delle funzionalità richieste. Ha il compito di redigere il documento di Analisi dei requisiti^G
Progettista	Il ruolo di progettista si occupa di tradurre i requisiti identificati dagli analisti in qualcosa di implementabile, e ne supervisiona l'implementazione da parte dei programmatori.
Programmatore	È la persona cui spetta il compito di tradurre le unità di design in codice funzionante. In concomitanza con il codice deve anche implementare i test automatici per la Verifica^G del corretto funzionamento del codice.
Verificatore	Ha il compito di assicurarsi che le attività vengano svolte correttamente seguendo i processi di Ciclo di vita^G . Si occupa di eseguire test approfonditi e revisioni sul software. Ha il compito di verificare le modifiche apportate ai vari documenti ed eventualmente esponendo le criticità che presentano; deve perciò avere un'approfondita conoscenza dei requisiti per ogni Processo^G di vita.

Tabella 3: Descrizione dei ruoli del progetto e delle loro responsabilità



4.1.4 Struttura dei team GitHub

L'organizzazione interna del gruppo (si veda sezione 4.1) è stata implementata tramite la funzione nativa di GitHub Teams, che consente di suddividere i membri in sotto-gruppi corrispondenti ai ruoli previsti dal progetto (si veda Sezione 4.1.3). Ogni ruolo forma un team composto da uno a più membri, così da distribuire responsabilità e carico di **Verifica^G**.

Le figure che interessano le **Pull Request^G** sono principalmente due:

- verificatori, scelti tra i membri del team con responsabilità di **Verifica^G** in quello **Sprint^G**, devono controllare la correttezza formale delle modifiche;
- responsabile, una volta ottenuta l'approvazione dei verificatori, è incaricato di completare l'operazione di **Merge^G** sul **branch main** nel rispetto delle regole di protezione del **Branch^G**.

Questa struttura garantisce che ogni modifica venga validata da più persone e che il **Processo^G** rispetti il **Processo^G** di qualità (si veda Sezione 3.3).

4.1.5 Coordinamento

Il coordinamento del gruppo è garantito attraverso riunioni periodiche, interne ed esterne, finalizzate a mantenere un allineamento continuo sullo stato dei lavori, sui problemi emersi e sulle decisioni organizzative che emergono in fase di lavoro.

4.1.5.1 Riunioni Le riunioni previste dal **Processo^G** di gestione dei processi sono suddivise in due categorie principali:

- **Interne:** si svolgono tra i soli membri del gruppo e hanno luogo principalmente su Discord. Ogni **Sprint^G** ha una durata di due settimane e prevede tre incontri:
 - uno a fine **Sprint^G**, comprendente **Sprint Review^G** **Sprint Retrospective^G** e **Sprint^G** Planning. In questa sede il **Responsabile Verifica^G** lo stato delle attività concluse, analizza eventuali problemi riscontrati e pianifica il lavoro del successivo **Sprint^G**;
 - due durante lo svolgimento dello **Sprint^G**, in cui vengono trattati i temi dell'ordine del giorno e ciascun membro espone le attività completate, quelle ancora in corso e le difficoltà incontrate.

Per ogni riunione vengono assegnati specifici ruoli:

- **Scriba:** i membri incaricati di prendere appunti durante gli incontri;
- **Portavoce:** identificato nel responsabile, che modera e coordina la discussione;



– **Redattore**: responsabile della stesura del **Verbale^G** e del caricamento nel **Repository^G**.

- **Esterne**: coinvolgono anche figure esterne quali i docenti o l'azienda **Proponente^G Miriade S.r.l.**. Queste riunioni vengono svolte generalmente tramite Google Meet. La frequenza concordata è di un incontro al mese, con la possibilità di richiederne ulteriori qualora fossero necessari chiarimenti o approfondimenti specifici.

L'esito di ogni incontro, interno o esterno, deve essere registrato mediante la redazione di un **Verbale^G**, secondo le modalità previste dal **Processo^G** di documentazione ([vedi Sezione 3.1](#)).

4.2. Infrastruttura

Il **Processo^G** di infrastruttura ha lo scopo di stabilire e mantenere l'infrastruttura necessaria per eseguire gli altri processi di **Ciclo di vita^G**.

L'infrastruttura, che sia hardware o software, comprende strumenti, standard e le strutture per lo sviluppo, il funzionamento e la manutenzione.

4.2.1 Attività previste

Il **Processo^G** di infrastruttura consiste delle seguenti attività:

- implementazione;
- istituzione;
- manutenzione.

4.2.2 Implementazione

L'infrastruttura operativa del progetto viene implementata in modo da soddisfare pienamente i requisiti dei processi che la utilizzano.

Ogni tecnologia impiegata viene scelta in base alla sua capacità di supportare efficacemente le attività del **Ciclo di vita^G**, migliorare la comunicazione tra i membri del team e garantire qualità, tracciabilità e mantenibilità dei prodotti generati.

La seguente sezione descrive gli strumenti adottati.

Strumento	Descrizione
-----------	-------------



Whatsapp	Piattaforma di messaggistica istantanea utilizzata per comunicazioni informali che non richiedono tracciamento. Scelta per la rapidità, l'immediatezza e la diffusione tra i membri del team. Facilita il coordinamento veloce su aspetti operativi di basso impatto.
Discord	Strumento di comunicazione tramite canali vocali e testuali, utilizzato per le riunioni interne che necessitano di tracciamento e organizzazione strutturata. Scelto per la stabilità, la possibilità di creare canali dedicati.
Google Meet	Piattaforma sincrona per videoconferenze utilizzata negli incontri con l'azienda Proponente^G , in quanto standard scelto dal Committente^G .
Google Chat	Piattaforma di messaggistica organizzata per stanze e conversazioni strutturate. Proposta dall'azienda Proponente^G come canale asincrono per le comunicazioni.
Git	Sistema di versionamento distribuito utilizzato per tracciare l'evoluzione del progetto, gestire i rami di sviluppo e mantenere l'integrità dello storico. Scelto per la diffusione nel settore e la capacità di supportare efficacemente lavori collaborativi.
GitHub	Piattaforma Cloud^G -based di hosting per Repository^G Git, utilizzata per la gestione collaborativa del codice. Scelta per le funzionalità integrate (Issue^G tracking, Pull Request^G , branching rules, code review) che supportano il Processo^G di sviluppo.
L ^A T _E X	Linguaggio di markup per la produzione di documenti tecnici e accademici. Scelto per l'elevata qualità tipografica e per la possibilità di definire template uniformi, garantendo coerenza e mantenibilità della documentazione.
Overleaf	Piattaforma di scrittura collaborativa online per file LaTeX. Utilizzata nelle fasi iniziali per l'apprendimento della sintassi e per la stesura rapida, prima di passare al workflow locale/Git.
Google Mail	Servizio di posta elettronica utilizzato come canale asincrono formale di comunicazione verso l'esterno. Scelto per la sua affidabilità e per l'integrazione con gli altri strumenti Google.
Google Drive	Spazio di archiviazione Cloud^G utilizzato dal gruppo per raccogliere documenti correlati, appunti delle riunioni, fogli di calcolo in cui il gruppo registra le informazioni informali riguardanti i periodi di Sprint^G . Scelto per la centralizzazione dei contenuti, l'accessibilità da qualsiasi dispositivo e la possibilità di modifica concorrente.



Google Sheets	Foglio di calcolo online utilizzato per i dati riguardanti i singoli Sprint^G e rappresentarli in appositi grafici per una visualizzazione migliore. Scelto per la collaborazione sincrona e la facilità di aggiornamento in tempo reale.
Google Docs	Editor di testi online per la stesura collaborativa di contenuti non ufficiali come bozze interne per i verbali. Scelto per la semplicità d'uso e la modifica concorrente.
Google Presentazioni	Strumento per la realizzazione di presentazioni utilizzate nelle esposizioni verso professori e Stakeholder^G . Scelto per la capacità di collaborazione e per i template professionali disponibili.
Script Python	Script sviluppati per automatizzare attività ricorrenti nel Processo^G , come operazioni di generazione automatica o Validazione^G . Scelti per la flessibilità del linguaggio e la facilità di integrazione con altre tecnologie.
Lucidchart	Strumento per la creazione di diagrammi, utilizzato per la redazione dei diagrammi UML dei casi d'uso. Scelto per la possibilità di collaborazione in tempo reale, che ha consentito a più membri del gruppo di lavorare simultaneamente.

Tabella 4: Strumenti utilizzati per l'infrastruttura

4.2.3 Istituzione

La presente attività di **istituzione dell'infrastruttura** ha lo scopo di documentare la configurazione dell'infrastruttura per facilitarne la manutenzione e la modifica da parte dei membri del gruppo BitByBit.

4.2.3.1 Discord Discord è utilizzato come strumento per gli incontri del team e come archivio centralizzato delle risorse condivise, tra cui file e link relativi allo sviluppo e al materiale del corso. Il gruppo BitByBit ha organizzato la piattaforma tramite più canali testuali, suddivisi per tipologia di contenuto (risorse di sviluppo, file e chat generale), e un canale vocale dedicato allo svolgimento delle riunioni.

4.2.3.2 Git Per garantire coerenza operativa tra tutti i membri del team, è previsto un insieme minimo di configurazioni locali che ciascun componente deve applicare al proprio ambiente Git.



- Ogni membro deve definire il proprio nome e indirizzo e-mail in Git, così che ogni commit sia attribuito in modo univoco e riconoscibile all'interno dello storico del progetto.
- Il membro deve assicurarsi che Git sia correttamente installato nel proprio ambiente, e che sia possibile accedere alla **Repository**^G remota tramite le chiavi SSH configurate.

4.2.3.3 GitHub La configurazione del **Repository**^G è definita per supportare il workflow adottato e garantire qualità, tracciabilità e sicurezza delle modifiche.

Al suo interno è possibile trovare varie directories, di seguito verranno descritte le più rilevanti.

- **.github** contiene la configurazione delle automazioni del progetto. Include il file **CODEOWNERS**, che definisce i responsabili delle diverse directory e consente a GitHub di assegnare automaticamente i revisori per le **Pull Request**^G. Contiene inoltre il file **latex_template.txt**, che rappresenta il template comune a tutti i documenti del gruppo BitByBit, e **template_pull_request.md**, il modello utilizzato per la creazione automatica delle **Pull Request**^G.
 - **.github/workflows** contiene i workflow **GitHub Actions**^G dedicati all'integrazione continua e all'automazione delle attività del **Repository**^G.
 - **.github/scripts** contiene gli script Python incaricati dell'automazione dei processi, come la compilazione dei file **.tex** e l'inserimento automatico del template **L^AT_EX** durante la creazione di nuovi documenti.
- **website** raccoglie gli elementi necessari alla pubblicazione del sito del progetto, tra cui fogli di stile e risorse grafiche.
- **resources** funge da archivio centralizzato delle risorse condivise, includendo immagini, loghi e altri materiali comuni utilizzati nel **Repository**^G.
- **src** contiene tutti i file sorgente **.tex**, ovvero il codice **L^AT_EX** prodotto dal gruppo BitByBit, organizzato in sezioni corrispondenti alle diverse fasi del progetto.
- **docs** replica la struttura della directory **src**, ma raccoglie i file **PDF** generati a partire dai documenti **L^AT_EX**.

4.3. Processo di miglioramento

Il **Processo**^G di miglioramento è il **Processo**^G tramite il quale vengono analizzati, valutati, misurati e migliorati i processi del **Ciclo di vita**^G.



4.3.1 Scopo

Durante tutta la fase di lavoro il team si pone l'obiettivo di seguire i principi del **miglioramento continuo**. Lo scopo principale è identificare problematiche ricorrenti in processi, ruoli e situazioni dell'intero **Ciclo di vita^G** in modo da poterli evitare in futuro. Dopo aver identificato tali punti critici, il team si impegna a discuterne insieme per trovare spunti di miglioramento da trasformare in **processi standardizzati**.

4.3.2 Descrizione

Il miglioramento continuo rappresenta un ciclo iterativo che consente al team di adattarsi in maniera flessibile alle esigenze in evoluzione del progetto, assicurando progressi costanti e un incremento dell'efficienza operativa. Ogni fase del **Ciclo di vita^G** del progetto, dalla definizione delle attività alla redazione dei documenti, riflette questo approccio.

4.3.3 Implementazione

Durante l'esecuzione delle attività e la stesura dei documenti, il team applica sistematicamente il principio del **miglioramento continuo**. In caso di riscontro di un problema, viene convocata una riunione interna in cui tutti i membri del team discutono l'argomento, valutando soluzioni possibili e alternative. L'applicazione pratica del miglioramento viene registrata nella sezione **“Decisioni”** del corrispondente **Verbale^G** interno, comprensiva della giustificazione della scelta e del consenso tra tutti i membri del team.

Qualora la decisione comporti modifiche tecniche, è prevista la creazione di una **Issue^G** specifica, secondo quanto stabilito dalle Norme di Progetto. La **Issue^G** dovrà riportare l'etichetta “enhancement” e contenere tutte le informazioni necessarie a descrivere il cambiamento, le motivazioni alla base della decisione e l'impatto previsto sulle attività o sul software.

Inoltre, tutte le problematiche rilevate e le soluzioni adottate continuano a essere documentate e analizzate, garantendo una gestione consapevole e mirata delle criticità lungo l'intero **Processo^G** di sviluppo. Questo approccio strutturato evita il ripetersi di errori già affrontati e favorisce il consolidamento di pratiche efficaci e condivise.

4.4. Processo di formazione

Il **Processo^G** di formazione è il **Processo^G** che garantisce che il team abbia le conoscenze tecniche necessarie per svolgere il lavoro che deve portare a termine.

4.4.1 Scopo

Lo scopo del **Processo^G** di formazione è garantire che ogni membro del team sia adeguatamente formato e competente per lo svolgimento delle attività di progetto. La formazione



è considerata un fattore essenziale per assicurare la corretta acquisizione, sviluppo, utilizzo e manutenzione del prodotto software, nonché per supportare efficacemente le attività di gestione e di natura tecnica.

4.4.2 Descrizione

Il **Processo^G** di formazione prevede l'**Analisi dei requisiti^G** di progetto al fine di individuare le competenze necessarie, i ruoli coinvolti e i livelli di conoscenza richiesti. Sulla base di tale analisi, vengono definite le tipologie di formazione e le categorie di personale che necessitano di istruzione. La formazione viene pianificata e avviata sin dalle prime fasi del progetto, in modo da garantire la disponibilità tempestiva di risorse qualificate durante l'intero **Ciclo di vita^G** del software.

4.4.3 Aspettative formative

Il **Processo^G** di formazione è finalizzato a mettere ciascun membro del gruppo nelle condizioni di:

- Ottenere dimestichezza con Git, GitHub e altri strumenti di controllo versione in uso durante il progetto.
- Acquisire competenze di scrittura con $\text{\LaTeX}$ per la redazione di documenti con formattazione e impaginazione adeguati.
- Acquisire competenze con i linguaggi di programmazione, le piattaforme, i programmi, le librerie, gli strumenti e gli ambienti di sviluppo necessari per la realizzazione del prodotto software concordato.

4.4.4 Piano di formazione

Ciascun membro del team è tenuto a studiare in maniera autonoma gli argomenti d'interesse. L'intero team stabilisce cosa studiare durante gli incontri interni, creando **Issue^G** dedicate assegnate a ciascun membro del team. Ogni membro è tenuto ad avere una visione d'insieme delle tecnologie e delle metodologie necessarie per il completamento del progetto, ma ha completa libertà per quanto riguarda i metodi di apprendimento che preferisce utilizzare. Ciascun membro è tenuto ad approfondire argomenti specifici sulla base dei compiti che gli sono stati assegnati. Al completamento di tali compiti e al termine di ogni **Sprint^G**, i membri devono confrontarsi condividendo in maniera sintetica ciò che hanno imparato relativamente ai compiti che hanno portato a termine o sui quali hanno fatto progressi significativi. Questo approccio mira ad alleggerire il carico globale di studio, a garantire una maggiore velocità esecutiva e a promuovere una formazione meno teorica e più incentrata sulla comprensione globale del sistema sul quale si lavora.



5. Metriche

Le seguenti metriche compongono il manuale delle metriche che il gruppo ha identificato per questo specifico progetto. Le metriche identificate dipendono dal preciso software che si deve sviluppare e dalle tecnologie che vengono utilizzate.

5.1. Nomenclatura

La nomenclatura utilizzata per le metriche è la seguente:

$$M-<\text{tipo}>X$$

dove:

- M: indica la sigla per **Metrica^G**;
- <tipo>: indica che aspetto misura la **Metrica^G**;
 - PD per Prodotto;
 - PC per **Processo^G**;
- X: un numero incrementale per identificare unicamente la **Metrica^G**. Il conteggio è separato per le 2 tipologie.

5.2. Metriche di Prodotto

La qualità del software dipende da un insieme di attributi che caratterizzano il comportamento, la struttura e l'evolvibilità del sistema nel tempo.

Gli attributi di qualità considerati nel progetto sono i seguenti:

- **complessità (complexity)**, rappresenta il grado di articolazione logica e strutturale del sistema;
- **comprensibilità (understandability)**, indica la facilità con cui il software può essere letto e compreso;
- **manutenibilità (maintainability)**, rappresenta la capacità del prodotto software di essere modificato, corretto o esteso;
- **efficienza (efficiency)**, indica la capacità del sistema di fornire le funzionalità richieste utilizzando in modo ottimale le risorse disponibili;
- **affidabilità (reliability)**, rappresenta la capacità del sistema di mantenere un livello di prestazioni accettabile quando utilizzato in condizioni specificate.

Le metriche di prodotto adottate nel progetto sono suddivise nelle seguenti due categorie:



- **Metriche dinamiche**, raccolte mediante misurazioni effettuate su un programma in esecuzione. Tali metriche supportano la valutazione dell'efficienza e dell'affidabilità del sistema;
- **Metriche statiche**, raccolte mediante misurazioni effettuate su rappresentazioni del sistema, quali codice sorgente, progetto o documentazione. Esse supportano la valutazione della complessità, della comprensibilità e della manutenibilità del sistema o dei suoi componenti.

5.2.1 Metriche statiche

5.2.1.1 Requisiti obbligatori soddisfatti

- **Codice:** M-PD01
- **Formula:** $\frac{\# \text{ di requisiti obbligatori soddisfatti}}{\# \text{ di requisiti obbligatori totali}} * 100$
- **Descrizione:** fornisce un'indicazione del grado di requisiti obbligatori che sono stati implementati nel prodotto software fino a quel momento.

5.2.1.2 Requisiti desiderabili soddisfatti

- **Codice:** M-PD02
- **Formula:** $\frac{\# \text{ di requisiti desiderabili soddisfatti}}{\# \text{ di requisiti desiderabili totali}} * 100$
- **Descrizione:** fornisce un'indicazione del grado di requisiti desiderabili che sono stati implementati nel prodotto software fino a quel momento.

5.2.1.3 Requisiti opzionali soddisfatti

- **Codice:** M-PD03
- **Formula:** $\frac{\# \text{ di requisiti opzionali soddisfatti}}{\# \text{ di requisiti opzionali totali}} * 100$
- **Descrizione:** fornisce un'indicazione del grado di requisiti opzionali che sono stati implementati nel prodotto software fino a quel momento.

5.2.1.4 Reliability Rating (RR)

- **Codice:** M-PD04
- **Formula:** assegnata automaticamente da SonarQube secondo la scala:
 - **A** → 0 bug
 - **B** → almeno 1 bug minore



- **C** → almeno 1 bug maggiore
 - **D** → almeno 1 bug critico
 - **E** → almeno 1 bug bloccante
- **Descrizione:** misura l'affidabilità del codice sulla base della presenza e gravità dei bug rilevati dall'**Analisi Statica**^G. Fornisce un indicatore immediato della qualità del software dal punto di vista dell'affidabilità.

5.2.1.5 Maintainability Rating

- **Codice:** M-PD05
- **Formula:** $DR = \frac{\text{Technical Debt}}{\text{cost_per_line} \times NCLOC} \cdot 100$, assegnata automaticamente da SonarQube secondo la scala:
 - **A** → $0\% \leq DR \leq 5\%$
 - **B** → $5\% \leq DR < 10\%$
 - **C** → $10\% \leq DR < 20\%$
 - **D** → $20\% \leq DR < 50\%$
 - **E** → $DR \geq 50\%$
- **Descrizione:** misura la manutenibilità del codice tramite un rating, calcolato a partire dal *Debt Ratio* (DR), ovvero il rapporto tra il *Technical Debt* (stima in minuti del costo per correggere tutti i problemi di manutenibilità rilevati) e il costo stimato per sviluppare l'intero codice, dove il costo per linea è predefinito a 30 minuti sonarqube-metrics.

5.2.1.6 Cyclomatic Complexity (CYC)

- **Codice:** M-PD06
- **Formula:** $CC = 1 + \#\text{rami condizionali}$
- **Descrizione:** misura il numero di percorsi linearmente indipendenti nel flusso di controllo del codice. Il valore viene calcolato a livello di funzione incrementando un contatore di uno ogni volta che il flusso di controllo si divide generando un nuovo ramo condizionale.

5.2.1.7 Cognitive Complexity (COC)

- **Codice:** M-PD07
- **Formula:** $COC = \sum_{i=1}^n w_i$



- **Descrizione:** misura quanto è difficile comprendere il flusso di controllo del codice. Penalizza in modo progressivo l'annidamento delle strutture condizionali (**if**, **else**, **switch**, cicli): ogni livello aggiuntivo di annidamento aumenta il peso w_i del costruito.

5.2.2 Metriche dinamiche

5.2.2.1 Code Coverage (CC)

- **Codice:** M-PD08
- **Formula:** $CC = \frac{CT + LC}{B + EL} \cdot 100$
 - CT = numero di condizioni valutate come vere almeno una volta
 - LC = linee coperte dai test ($LC = \#$ linee eseguibili – $\#$ linee non coperte)
 - B = numero totale di condizioni
 - EL = numero totale di linee eseguibili
- **Descrizione:** fornisce una stima accurata di quanta parte del codice è stata effettivamente verificata dai test.

5.2.2.2 Unit Test Success Density (UTSD)

- **Codice:** M-PD09
- **Formula:** $UTSD = \frac{\# \text{ test} - (\# \text{ test errors} + \# \text{ test failures})}{\# \text{ test}} \cdot 100$
- **Descrizione:** misura la percentuale di test unitari eseguiti con successo. Vengono considerati come esito negativo sia i test che non superano le condizioni di **Verifica^G** e le asserzioni previste (*test failures*), sia quelli che si interrompono anomaliamente a causa di un'eccezione imprevista (*test errors*).

5.3. Metriche di Processo

5.3.1 Fornitura

5.3.1.1 Budget at Completion (BAC)

- **Codice:** M-PC01
- **Formula:** BAC = budget totale pianificato per il progetto
- **Descrizione:** rappresenta il costo totale pianificato del progetto al momento della definizione del piano. Rappresenta una **Metrica^G** di riferimento per altre misurazioni.



5.3.1.2 Planned Value (PV)

- **Codice:** M-PC02
- **Formula:** $PV = (BAC) \cdot (\% \text{ di lavoro pianificata nello Sprint}^G)$
- **Descrizione:** rappresenta il preventivo per i costi del lavoro che si intende eseguire durante uno **Sprint**^G di progetto. La percentuale di lavoro pianificato corrisponde a $\frac{\text{ore previste in uno Sprint}^G}{\text{ore totali}}$. **Ore totali** corrisponde alla somma delle ore che da progetto ogni componente del gruppo deve eseguire.

5.3.1.3 Actual Cost (AC)

- **Codice:** M-PC03
- **Formula:** $AC = \text{costo effettivo sostenuto in uno Sprint}^G$
- **Descrizione:** rappresenta i costi effettivamente sostenuti per completare il lavoro svolto durante uno **Sprint**^G di progetto.

5.3.1.4 Earned Value (EV)

- **Codice:** M-PC04
- **Formula:** $EV = \frac{\# \text{ di Issue}^G \text{ completate}}{\# \text{ di Issue}^G \text{ pianificate}} \cdot PV$
- **Descrizione:** rappresenta il valore del lavoro realmente completato rispetto a quanto pianificato. Consente di valutare l'avanzamento effettivo del progetto indipendentemente dai costi sostenuti.

5.3.1.5 Schedule Performance Index (SPI)

- **Codice:** M-PC05
- **Formula:** $SPI = \frac{EV}{PV}$
- **Descrizione:** misura l'efficacia della gestione del tempo. Esso confronta la quantità di lavoro effettivamente completata con la quantità di lavoro pianificata. Un valore = 1 indica che il progetto è in linea con il piano stabilito, un valore < 1 evidenzia un ritardo.

5.3.1.6 Cost Performance Index (CPI)

- **Codice:** M-PC06
- **Formula:** $CPI = \frac{EV}{AC}$
- **Descrizione:** misura l'efficienza con cui il progetto utilizza il proprio budget. Un valore ≥ 1 indica che si sta rispettando il budget prefissato.



5.3.1.7 Estimate at Completion (EAC)

- **Codice:** M-PC07

- **Formula:** $EAC = ETC + \sum_{i=1}^n AC_i$

- **Descrizione:** stima il costo totale del progetto al completamento, assumendo che l'andamento futuro segua l'efficienza attuale. Viene confrontato con il Budget at Completion per individuare potenziali scostamenti. La sommatoria viene calcolata sull'Actual Cost di ogni Sprint concluso.

5.3.1.8 Estimate to Complete (ETC)

- **Codice:** M-PC08

- **Formula:** $ETC = \frac{BAC - \sum_{i=1}^n EV_i}{CPI}$

- **Descrizione:** misura il costo previsto per completare il lavoro rimanente del progetto escludendo i costi già sostenuti. La sommatoria si riferisce

5.3.2 Documentazione

5.3.2.1 Indice di Gulepase

- **Codice:** M-PC09

- **Formula:** $89 + \frac{300 \cdot (\text{numero di frasi}) - 10 \cdot (\text{numero di lettere})}{\text{numero di parole}}$

- **Descrizione:** indice di leggibilità tarato su testi in lingua italiana. In generale risulta che testi con un indice:
 - inferiore a 80 sono difficili da leggere per chi ha la licenza elementare
 - inferiore a 60 sono difficili da leggere per chi ha la licenza media
 - inferiore a 40 sono difficili da leggere per chi ha un diploma superiore

5.3.3 Gestione dei processi

5.3.3.1 Sprint Commitment Reliability (SCR)

- **Codice:** M-PC10

- **Formula:** $\frac{\# \text{Issue}^G \text{ completate nello Sprint}^G}{\# \text{Issue}^G \text{ pianificate}} \cdot 100$

- **Descrizione:** misura la capacità del team di rispettare gli impegni pianificati per lo Sprint^G. Supporta la valutazione della qualità della pianificazione e della stabilità del Processo^G di sviluppo.